# Hospital Management System

A DBMS Capstone Project Report submitted in partial fulfillment of the requirements for the award of the degree of

**BACHELOR OF TECHNOLOGY**

in

**ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**


Submitted By

**D.R.PALGUN – 25B11AI241**

**K.ASHOK – 25B11AI478**

**P.V.S.ROHITH REDDY – 25B11AI892**

**G.M.SRIRAM – 25B11AI311**

Under the esteemed guidance of

**Mr. T.Srinivasulu** M.Tech, (Ph.D.)

**Assistant Professor**






**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

**ADITYA UNIVERSITY**

(Formerly Aditya Engineering College (A))

Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India


**2026 – 2027**

ADITYA UNIVERSITY
(Formerly Aditya Engineering College(A))
Aditya Nagar, ADB Road, Surampalem, Andhra Pradesh, India

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**




# CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled "**Hospital Management System**" is being submitted by **D.R.PALGUN** (Roll No. **25B11AI241**), **K.ASHOK** (Roll No.**25B11AI478**), **P.V.S ROHITH REDDY** (Roll No. **25B11AI892**) and **G.M.SRIRAM** (Roll No. **25B11AI311**) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the **Capstone Project** for the course **Database Management Systems (2501IT05)** during the academic year **2026–2027**, and records the database design, implementation, and documentation undertaken by the team.

**Project Guide**
Mr. T. Srinivasulu, M. Tech, (Ph.D.)
Associate professor

Head of the Department
Dr. K. Naga Bhargavi, M. Tech, Ph.D.
Associate Professor & HOD

**External Examiner**

# DECLARATION

We hereby declare that the DBMS Capstone Project entitled "**Hospital Management System**" is a genuine work carried out by us. The database design, SQL implementation, queries and documentation presented in this report are our own work, completed under the guidance of, **T.Srinivasulu** sir, Department of **Artificial Intelligence and Machine Learning**, Aditya University, Surampalem. We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.


**Project Associates**

D.R.PALGUN (25B11AI241)

K.ASHOK (25B11AI478)

P.V.S.ROHITH REDDY (25B11AI892)

G.M.SRIRAM (25B11AI311)


Date: _____

Place: Surampalem

# ACKNOWLEDGEMENT

It is with immense pleasure that we express our indebted gratitude to our project guide T.SRINIVASULU sir, Department of Artificial Intelligence & Machine Learning, who guided us throughout the capstone project. We are thankful for the constant encouragement and valuable suggestions received at every review stage.

We are grateful to Dr.K.NagaBhargavi, Head of the Department, for inspiring us and for arranging the facilities and resources needed for this project.

We extend our sincere thanks to the Management, Dr. M. Sreenivasa Reddy, Deputy Pro-Chancellor, Dr. M. B. Srinivas, Vice-Chancellor, and Dr. A. Ramesh, Pro Vice-Chancellor (Engineering & Sciences), for providing an excellent academic environment.

Not to forget, we thank the Teaching and Non-Teaching Staff, our parents and our friends, who have directly or indirectly supported us in completing this project on time.

**Project Associates**

Student Name: D.R.PALGUN Regd. No. 25B11AI241

Student Name: K.ASHOK Regd. No. 25B11AI478

Student Name: P.V.S.ROHITH REDDY Regd. No. 25B11AI892

Student Name:G.M.SRIRAM Regd. No. 25B11AI311

# ADITYA UNIVERSITY

Surampalem

Department of Computer Science and Engineering

# DBMS CAPSTONE PROJECT

## Project Documentation Report

**Submitted by:**

| S.No | Student Name | Roll Number |
|---|---|---|
| 1 | D.R.PALGUN | 25B11AI241 |
| 2 | K.ASHOK | 25B11AI478 |
| 3 | P.V.S.ROHITH REDDY | 25B11AI892 |
| 4 | G. M.SRIRAM | 25B11AI311 |

Course: Database Management System (DBMS)  Course Code: DBMS

Faculty Coordinator:

Academic Year: 2026–27  Date of Submission: 14 September 2026

GitHub Repository Link: https://github.com/palgun9999/Hospital-Management-System

# ABSTRACT

The **Hospital Management System** is a database project developed to efficiently manage the day-to-day information and operations of a hospital. Hospitals handle large amounts of data related to patients, doctors, departments, appointments, treatments, admissions, medicines, prescriptions, billing, and payments. Managing such information using manual records or spreadsheets can result in data redundancy, inconsistency, difficulty in retrieval, and poor data integrity. The purpose of this project is to develop a centralized relational database that provides accurate storage, easy retrieval, and effective management of hospital information. The database consists of 15 entities, with major tables including **Department, Doctor, Patient, Appointment, Treatment, Ward, Room, Admission, Medical_Record, Prescription, Medicine, Bill,** and **Payment.** The system is implemented using MySQL and SQL-based database tools. SQL operations such as CREATE, INSERT, UPDATE, SELECT, WHERE, ORDER BY, aggregate functions, primary keys, foreign keys, UNIQUE constraints, and CHECK constraints are implemented. The database supports patient registration, doctor and department management, treatment records, prescriptions, room allocation, billing, and payments. The resulting system reduces data redundancy, improves consistency and integrity, simplifies information retrieval, and provides an organized foundation for efficient hospital data management.

ADITYA
UNIVERSITY

# INDEX


| Chapter | Title | Page No. |
|---|---|---|
| **1** | **Introduction** | |
| | 1.1 Introduction to the Domain | |
| | 1.2 Problem Statement | |
| | 1.3 Motivation and Need for a Database | |
| | 1.4 Objectives of the Project | |
| | 1.5 Scope of the Project | |
| | 1.6 Organization of the Report | |
| **2** | **System Study and Tools** | |
| | 2.1 Existing System and Its Limitations | |
| | 2.2 Proposed Database System | |
| | 2.3 Tools and Technologies Used | |
| | 2.4 Software and Hardware Requirements | |
| **3** | **Database Design (Review – I)** | |
| | 3.1 Identified Entities | |
| | 3.2 Entity Attributes and Attribute Types | |
| | 3.3 ER Diagram | |
| | 3.4 Relationships and Cardinality | |
| | 3.5 Relational Database Schema | |
| | 3.6 Primary Keys and Foreign Keys | |
| | 3.7 Normalization up to 3NF | |
| **4** | **Database Implementation (Review – II)** | |
| | 4.1 Database and Table Creation – DDL | |
| | 4.2 Sample Data Insertion – DML | |
| | 4.3 Constraints and Data Integrity | |
| | 4.4 Data Validation Rules | |
| **5** | **SQL Operations and Queries** | |
| | 5.1 DQL – Basic Retrieval Queries | |
| | 5.2 Join Operations | |
| | 5.3 Aggregate Functions, GROUP BY and HAVING | |
| | 5.4 Nested and Correlated Subqueries | |
| | 5.5 Views and Relational Set Operations | |
| | 5.6 DCL and TCL Operations | |
| **6** | **Modules and Functionalities** | |
| **7** | **Results – Sample Outputs and Screenshots** | |
| **8** | **Testing and Data Validation** | |
| **9** | **GitHub Repository and Team Contribution** | |
| **10** | **Conclusion and Future Enhancements** | |
| | References | |
| | Appendix A – Complete SQL Script | |

# LIST OF FIGURES


| Figure No. | Title of the Figure | Page No. |
|---|---|---|
| Fig. 3.1 | ER Diagram of the Hospital Management System | 10 |
| Fig. 3.2 | Relational Schema Diagram | 15 |
| Fig. 4.1 | Successful Creation of Tables | 20 |
| Fig. 4.2 | Sample Data Inserted into the Tables | 22 |
| Fig. 4.3 | Referential Integrity Enforced by the Database | 23 |
| Fig. 7.1 | Doctor whose experience greater than 7 years | 30 |
| Fig. 7.2 | Medicines price greater than 30 | 30 |
| Fig. 7.3 | Average Experience | 31 |
| Fig. 7.4 | List of all the treatments | 31 |


# LIST OF TABLES


| Table No. | Title of the Table | Page No. |
|---|---|---|
| Table 2.1 | Tools and Technologies Used | 6 |
| Table 2.2 | Software and Hardware Requirements | 6 |
| Table 3.1 | Entities Identified for the Selected Domain | 7 |
| Table 3.2 | Attributes of Each Entity | 7 |
| Table 3.3 | Relationships and Cardinality | 11 |
| Table 3.4 | Relational Database Schema | 11 |
| Table 3.5 | Primary Key and Foreign Key Definitions | 15 |
| Table 3.6 | Normalization Steps | 16 |
| Table 4.1 | Sample Data Summary | 20 |
| Table 4.2 | Constraints and Data Integrity Rules | 22 |
| Table 5.1 | Basic Retrieval Queries | 24 |
| Table 5.2 | Join Operations | 25 |
| Table 5.4 | Subqueries | 26 |
| Table 5.5 | Views and Set Operations | 26 |
| Table 5.6 | DCL and TCL Operations | 27 |
| Table 6.1 | Modules and Their Functionalities | 29 |
| Table 8.1 | Test Cases and Results | 32 |
| Table 9.1 | Team Responsibilities | 34 |
| Table 9.2 | Review-wise Progress Record | 34 |

# CHAPTER 1: INTRODUCTION

## 1.1 Introduction to the Domain

The selected domain for this project is the **Hospital Management System** . A hospital is a healthcare organisation that provides medical services such as patient consultation, diagnosis, treatment, admission, medication, and emergency care. It consists of multiple departments, wards, rooms, doctors, and other healthcare resources that work together to provide effective patient care. Since many hospital activities are interconnected, managing the related information in an organised and accurate manner is essential for smooth daily operations.

On a day-to-day basis, the hospital handles a large amount of information, including patient personal and contact details, doctor information and specializations, departments, appointments, symptoms, diagnoses, treatments, medical records, prescriptions, medicines, wards, rooms, and admissions. The organisation also maintains financial information such as consultation fees, medicine charges, bed charges, bills, payment status, and payment transactions. This information must be stored systematically so that authorised users can retrieve and update records whenever required.

The proposed HMS database brings these different types of information together in a structured relational database. By connecting patients with doctors, appointments, treatments, prescriptions, admissions, and bills, the system provides an organised approach to hospital data management and supports efficient healthcare and administrative activities.

## 1.2 Problem Statement

In many hospitals, patient and administrative information is traditionally maintained using paper files, registers, spreadsheets, or separate software applications for different departments. Patient details, doctor records, appointments, treatments, prescriptions, admissions, and billing information are often stored independently by various hospital units. As the volume of data grows, managing and updating records becomes increasingly difficult. Staff members may need to search through multiple documents or files to retrieve patient histories, treatment details, or billing records, resulting in delays and reduced operational efficiency.

This approach has several limitations. Since the same information may be entered in multiple places, data redundancy occurs, leading to unnecessary duplication of records. Manual updates can also create data inconsistencies, where different departments maintain conflicting versions of the same information. Retrieving specific records becomes time-consuming, especially when dealing with large amounts of data. Furthermore, traditional systems lack proper integrity checks, making it possible to enter invalid, incomplete, or duplicate data. Such issues can negatively affect patient care, reporting accuracy, and hospital administration.

The proposed Hospital Management System database addresses these problems by providing a centralized and integrated data management solution. Through the use of relational tables, primary keys, foreign keys, unique constraints, and validation rules, the system ensures data integrity and consistency. Information can be retrieved quickly using SQL queries, while redundancy is minimized through database normalization. The system enables efficient management of patients, doctors, appointments, treatments, prescriptions, admissions, and billing processes, resulting in improved accuracy, faster access to information, better resource utilization, and enhanced overall healthcare service delivery.

## 1.3 Motivation and Need for a Database

A Database Management System (DBMS) is essential for a Hospital Management System because hospitals generate and process large volumes of interconnected data every day. Information related to patients, doctors, departments, appointments, treatments, admissions, prescriptions, medicines, billing, and payments must be stored accurately and accessed quickly. Managing such complex and related data using ordinary files or spreadsheets becomes inefficient as the size of the organization grows.

File-based systems and spreadsheets have several limitations. They often lead to data redundancy because the same information may be stored in multiple files. This increases the chances of inconsistencies when records are updated in one place but not in others. Retrieving information across multiple files is time-consuming, and generating reports requires significant manual effort. Additionally, spreadsheets do not provide strong mechanisms for maintaining relationships between data, enforcing data integrity, or preventing duplicate and invalid entries.

A DBMS overcomes these limitations by providing a centralized and structured storage system. It uses primary keys, foreign keys, constraints, and normalization techniques to

maintain data accuracy and consistency. SQL queries enable fast retrieval, updating, and analysis of information, while integrity constraints ensure that only valid data is stored. Furthermore, a DBMS supports data security, multi-user access, backup and recovery mechanisms, and efficient management of large datasets. For a hospital environment where accurate and timely information is critical for patient care and administrative operations, a DBMS provides a more reliable, scalable, and secure solution than traditional files or spreadsheets.

## 1.4 Objectives of the Project

1. To design a normalized relational database for the selected domain with at least six entities.
2. To implement the database with appropriate primary keys, foreign keys and integrity constraints.
3. To store and manage patient, doctor, appointment, treatment, and billing information efficiently in a centralized database system.
4. To reduce data redundancy and improve data consistency through proper table relationships and normalization techniques.
5. To enable fast retrieval, updating, and reporting of hospital records using SQL queries and database management features.

## 1.5 Scope of the Project

The proposed Hospital Management System database covers the major operational and administrative activities performed within a hospital. It manages information related to departments, doctors, patients, patient contacts, appointments, treatments, medical records, prescriptions, medicines, wards, rooms, admissions, bills, and payments. The system supports patient registration, doctor and department management, appointment scheduling, treatment documentation, prescription generation, medicine inventory tracking, room allocation, patient admissions, billing, and payment processing. It also maintains relationships among these entities to ensure accurate and efficient data management.

Therefore, the scope of this project is limited to the core hospital operations required for managing patients, medical services, admissions, prescriptions, and financial transactions. These limitations help keep the database focused, manageable, and suitable for demonstrating relational database design concepts while meeting the requirements of the Hospital Management System project.

## 1.6 Organization of the Report

This report is organized into ten chapters. Chapter 2 presents the system study and the tools used. Chapter 3 covers the complete database design including the ER diagram, relational schema and normalization. Chapter 4 describes the database implementation with DDL and sample data. Chapter 5 presents the SQL operations and queries executed. Chapter 6 explains the modules and functionalities. Chapter 7 presents the results with sample outputs. Chapter 8 covers testing and data validation. Chapter 9 details the GitHub repository and the contribution of each team member. Chapter 10 concludes the report and outlines future enhancements.

# CHAPTER 2: SYSTEM STUDY AND TOOLS

## 2.1 Existing System and Its Limitations

Without a proper database system, hospital information is typically managed through paper records, registers, spreadsheets, or separate files maintained by different departments. Patient details, doctor information, appointment schedules, treatment records, prescriptions, admissions, and billing data are often stored independently. Hospital staff must manually record, update, and search for information whenever it is needed. As the number of patients and transactions increases, managing these records becomes more complicated and time-consuming.

This traditional approach has several limitations. First, it leads to data redundancy, as the same information may be entered multiple times in different files or departments. Second, data inconsistency can occur when changes made in one record are not reflected in all related records. Third, retrieving information becomes difficult because staff may need to search through numerous documents or spreadsheets to find specific details. Fourth, file-based systems lack proper integrity checks, making it possible to enter duplicate, incomplete, or incorrect data. In addition, generating reports, tracking patient history, and maintaining relationships between different records require significant manual effort.

A database-driven Hospital Management System overcomes these problems by storing all information in a centralized and structured manner. Through the use of relational tables, primary keys, foreign keys, and integrity constraints, the system minimizes redundancy, ensures consistency, supports fast data retrieval, and maintains accurate records. This results in improved efficiency, better data security, and more reliable hospital operations.

## 2.2 Proposed Database System

The proposed system is a Hospital Management System (HMS) based on a relational database management system. It centralizes all hospital-related information into a single integrated database that manages departments, doctors, patients, appointments, treatments, wards, rooms, admissions, medical records, prescriptions, medicines, bills, and payments. Each entity is stored in a separate table and linked through primary and foreign key relationships, ensuring that data is organized, consistent, and easy to manage. The system is implemented using MySQL and supports various SQL operations for data storage, retrieval, updating, and reporting.

Another major advantage is the ability to retrieve information quickly through SQL queries. Hospital staff can easily access patient histories, appointment details, treatment records, prescriptions, and billing information without searching through multiple files. The system also improves accuracy, saves time, enhances data security, supports efficient report generation, and enables better decision-making. Overall, the proposed database provides a reliable, scalable, and efficient solution for managing hospital operations and delivering quality healthcare services.

## 2.3 Tools and Technologies Used

**Table 2.1: Tools and Technologies Used**

| Category | Tool / Technology Used | Version / Remarks |
|---|---|---|
| Database Management System (DBMS) | MySQL | MySQL 8.0 or later |
| Database Server | MySQL Server | Used for database storage and management |
| Query and Administration Tool | MySQL Workbench | Used for writing, executing, and managing SQL queries |
| Query Language | SQL (Structured Query Language) | Used for database creation, manipulation, and retrieval |
| ER Diagram Tool | diagrams.net (draw.io) | Used for designing the ER diagram and database schema |
| Operating System | Windows 10/11 | Development environment |
| Version Control | GitHub | Used for source code and project version management |
| Documentation Tool | Microsoft Word | Used for preparing project report and documentation |
| Database Design Techniques | ER Modeling and Normalization | Used to reduce redundancy and improve data integrity |
| Development Environment | Localhost Environment | Used for testing and database implementation |

## 2.4 Software and Hardware Requirements

**Table 2.2: Software and Hardware Requirements**

| Requirement Type | Specification |
|---|---|
| Operating System | Windows 10 / Windows 11 |
| Database Management System | MySQL 8.0 or later |
| Database Administration Tool | MySQL Workbench |
| Query Language | SQL (Structured Query Language) |
| ER Diagram Tool | diagrams.net (draw.io) |
| Documentation Software | Microsoft Word |
| Processor | Intel Core i3 or higher / Equivalent AMD Processor |
| RAM | Minimum 4 GB (8 GB Recommended) |
| Storage | Minimum 20 GB Free Disk Space |
| Display | 1366 × 768 Resolution or Higher |
| Internet Connection | Required for software download, updates, and GitHub access |
| Version Control Tool | GitHub (Optional) |
| Input Devices | Keyboard and Mouse |

# CHAPTER 3: DATABASE DESIGN

## 3.1 Identified Entities

**Table 3.1: Entities Identified for the Selected Domain**

| S.No | Entity Name | Description |
|---|---|---|
| 1 | Department | Stores hospital department information |
| 2 | Doctor | Stores doctor details and specialization |
| 3 | Patient | Stores patient personal information |
| 4 | Patient_contact | Stores Patient contact numbers |
| 5 | Ward | Stores ward and Capacity details |
| 6 | Appointment | Stores room allocation information |
| 7 | Treatment | Manages patient doctor appointments |
| 8 | Bill | Records patient admission and discharge |
| 9 | Room | Maintains patient treatment information |
| 10 | Admission | Maintains patient medical records |
| 11 | Medical_Record | Stores prescription details and dates |
| 12 | Prescription | Stores medicine details and prices |
| 13 | Prescription_medicine | Links medicines with prescription |
| 14 | Medicine | Stores patient billing information |
| 15 | Payment | Records bill payment details |

## 3.2 Entity Attributes and Attribute Types

**Table 3.2: Attributes of Each Entity**

| Entity | Attribute | Data Type | Key (PK/FK) | Attribute Type |
|---|---|---|---|---|
| Department | dept_id | INT | PK | Simple |
| Department | dept_name | VARCHAR(100) | — | Simple |
| Department | floor_no | INT | — | Simple |
| Department | hod_id | INT | FK | Simple |
| Doctor | doctor_id | INT | PK | Simple |
| Doctor | first_name | VARCHAR(20) | — | Simple |
| Doctor | second_name | VARCHAR(20) | — | Simple |
| Doctor | specialization | VARCHAR(20) | — | Simple |
| Doctor | experience | INT | — | Simple |
| Doctor | email | VARCHAR(30) | — | Simple |
| Doctor | dept_id | INT | FK | Simple |
| Patient | patient_id | INT | PK | Simple |
| Patient | first_name | VARCHAR(50) | — | Simple |
| Patient | last_name | VARCHAR(50) | — | Simple |
| Patient | dob | DATE | — | Simple |
| Patient | blood_group | CHAR(5) | — | Simple |
| Patient | gender | VARCHAR(10) | — | Simple |
| Patient | address | TEXT | — | Simple |

| Patient_Contact | id | INT | PK | Simple |
|---|---|---|---|---|
| Patient_Contact | patient_id | INT | FK | Simple |
| Patient_Contact | contact | VARCHAR(15) | — | Simple |
| Ward | ward_id | INT | PK | Simple |
| Ward | ward_name | VARCHAR(50) | — | Simple |
| Ward | capacity | INT | — | Simple |
| Ward | dept_id | INT | FK | Simple |
| Room | room_id | INT | PK | Simple |
| Room | room_no | VARCHAR(20) | — | Simple |
| Room | room_type | VARCHAR(30) | — | Simple |
| Room | status | VARCHAR(20) | — | Simple |
| Room | ward_id | INT | FK | Simple |
| Appointment | appt_id | INT | PK | Simple |
| Appointment | patient_id | INT | FK | Simple |
| Appointment | doctor_id | INT | FK | Simple |
| Appointment | appt_date | DATE | — | Simple |
| Appointment | appt_time | TIME | — | Simple |
| Appointment | status | VARCHAR(20) | — | Simple |
| Appointment | symptoms | TEXT | — | Simple |
| Admission | admission_id | INT | PK | Simple |
| Admission | patient_id | INT | FK | Simple |
| Admission | room_id | INT | FK | Simple |
| Admission | admission_date | DATE | — | Simple |
| Admission | discharge_date | DATE | — | Simple |
| Treatment | treatment_id | INT | PK | Simple |
| Treatment | appt_id | INT | FK | Simple |
| Treatment | diagnosis | TEXT | — | Simple |
| Treatment | treatment_date | DATE | — | Simple |
| Treatment | ward_id | INT | FK | Simple |
| Medical_Record | record_id | INT | PK | Simple |
| Medical_Record | appointment_id | INT | FK | Simple |
| Medical_Record | diagnosis | TEXT | — | Simple |
| Medical_Record | treatment | TEXT | — | Simple |
| Medical_Record | record_date | DATE | — | Simple |
| Prescription | prescription_id | INT | PK | Simple |
| Prescription | record_id | INT | FK | Simple |
| Prescription | prescription_date | DATE | — | Simple |
| Medicine | med_id | INT | PK | Simple |
| Medicine | med_name | VARCHAR(100) | — | Simple |
| Medicine | description | TEXT | — | Simple |
| Medicine | price | DECIMAL(10,2) | — | Simple |
| Medicine | stock_quantity | INT | — | Simple |
| Prescription_Med | prescription_id | INT | PK, FK | Simple |
| Prescription_Med | med_id | INT | PK, FK | Simple |
| Prescription_Med | dosage | VARCHAR(50) | — | Simple |
| Prescription_Med | frequency | VARCHAR(50) | — | Simple |
| Prescription_Med | duration | VARCHAR(50) | — | Simple |
| Prescription_Med | quantity | INT | — | Simple |
| Bill | bill_id | INT | PK | Simple |
| Bill | patient_id | INT | FK | Simple |

| | | | | |
|---|---|---|---|---|
| Bill | treatment_id | INT | FK | Simple |
| Bill | consultation_fee | DECIMAL(10,2) | — | Simple |
| Bill | medicine_charges | DECIMAL(10,2) | — | Simple |
| Bill | bed_charges | DECIMAL(10,2) | — | Simple |
| Bill | payment_status | VARCHAR(20) | — | Simple |
| Bill | bill_date | DATE | — | Simple |
| Payment | payment_id | INT | PK | Simple |
| Payment | bill_id | INT | FK | Simple |
| Payment | payment_date | DATE | — | Simple |
| Payment | amount | DECIMAL(10,2) | — | Simple |
| Payment | payment_method | VARCHAR(30) | — | Simple |

## 3.3 ER Diagram

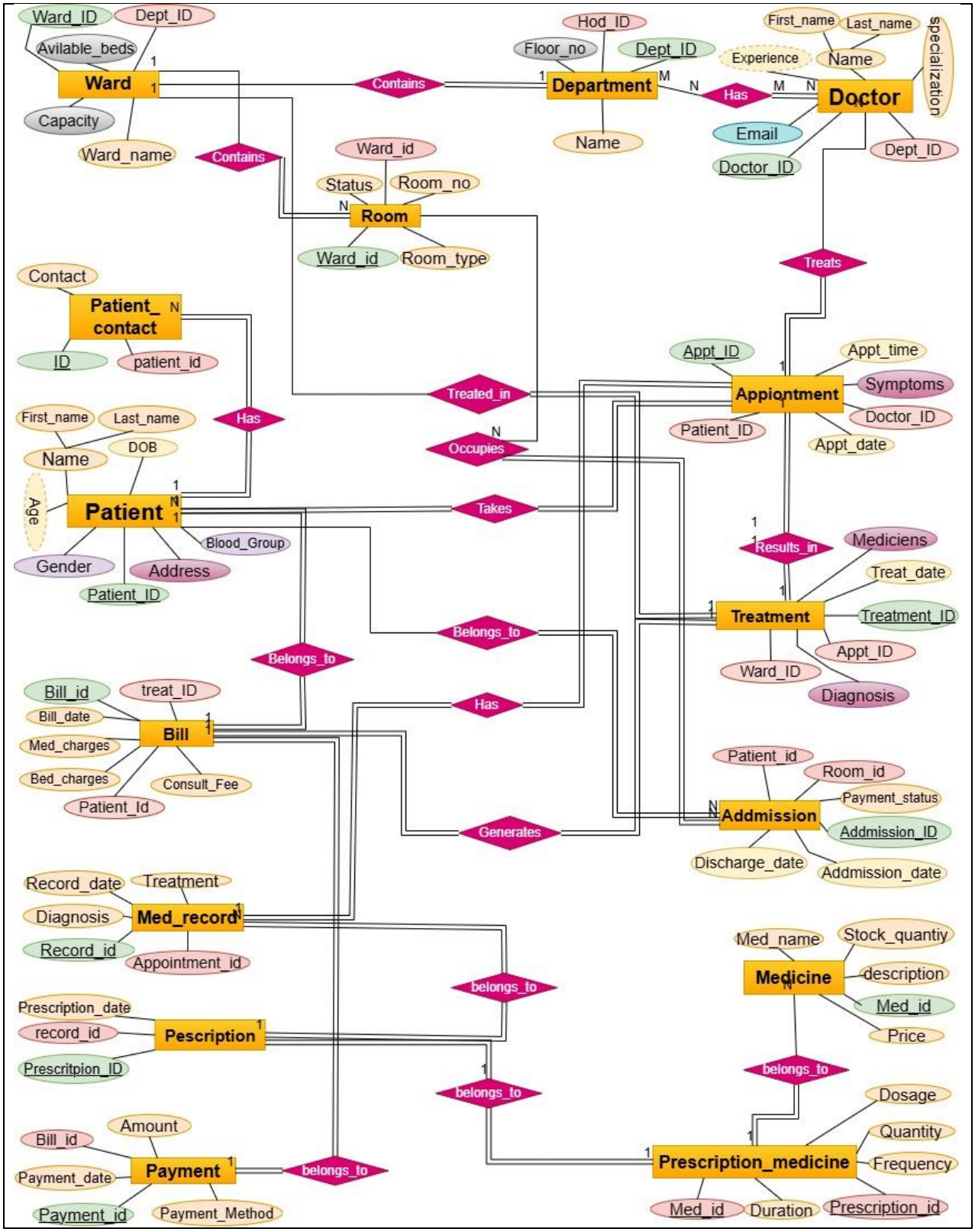



**Fig. 3.1: ER Diagram of the Hospital Management System**

## 3.4 Relationships and Cardinality

**Table 3.3: Relationships and Cardinality**

| Entity 1 | Relationship | Entity 2 | Cardinality (1:1 / 1:N / M:N) |
|---|---|---|---|
| Department | HAS | Doctor | 1:N |
| Department | CONTAINS | Ward | 1:N |
| Ward | CONTAINS | Room | 1:N |
| Patient | HAS | Patient_Contact | 1:N |
| Patient | BOOKS | Appointment | 1:N |
| Doctor | ATTENDS | Appointment | 1:N |
| Patient | HAS | Admission | 1:N |
| Room | ASSIGNED | Admission | 1:N |
| Appointment | RESULTS_IN | Treatment | 1:1 |
| Appointment | CREATES | Medical_Record | 1:1 |
| Medical_Record | HAS | Prescription | 1:N |
| Prescription | CONTAINS | Prescription_Med | 1:N |
| Medicine | INCLUDED_IN | Prescription_Med | 1:N |
| Patient | RECEIVES | Bill | 1:N |
| Treatment | GENERATES | Bill | 1:1 |
| Bill | HAS | Payment | 1:N |

## 3.5 Relational Database Schema

**Table 3.4: Relational Database Schema**

| Table Name | Column Name | Data Type | Key (PK / FK / —) | Constraints |
|---|---|---|---|---|
| Department | dept_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Department | dept_name | VARCHAR(100) | — | NOT NULL |
| Department | floor_no | INT | — | NOT NULL |
| Department | hod_id | INT | FK | REFERENCES Doctor(doctor_id) |

| | | | | |
|---|---|---|---|---|
| Doctor | doctor_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Doctor | first_name | VARCHAR(20) | — | NOT NULL |
| Doctor | last_name | VARCHAR(20) | — | NOT NULL |
| Doctor | specialization | VARCHAR(20) | — | NOT NULL |
| Doctor | experience | INT | — | NOT NULL |
| Doctor | email | VARCHAR(30) | — | UNIQUE |
| Doctor | dept_id | INT | FK | REFERENCES Department(dept_id) |
| Patient | patient_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Patient | first_name | VARCHAR(50) | — | NOT NULL |
| Patient | last_name | VARCHAR(50) | — | NOT NULL |
| Patient | dob | DATE | — | NOT NULL |
| Patient | blood_group | CHAR(5) | — | — |
| Patient | gender | VARCHAR(10) | — | — |
| Patient | address | TEXT | — | — |
| Patient_Contact | id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Patient_Contact | patient_id | INT | FK | REFERENCES Patient(patient_id) |
| Patient_Contact | contact | VARCHAR(15) | — | NOT NULL |
| Ward | ward_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Ward | ward_name | VARCHAR(50) | — | NOT NULL |
| Ward | capacity | INT | — | NOT NULL |
| Ward | dept_id | INT | FK | REFERENCES Department(dept_id) |
| Room | room_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Room | room_no | VARCHAR(20) | — | NOT NULL, UNIQUE |
| Room | room_type | VARCHAR(30) | — | NOT NULL |
| Room | status | VARCHAR(20) | — | NOT NULL |
| Room | ward_id | INT | FK | REFERENCES Ward(ward_id) |
| Appointment | appt_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Appointment | patient_id | INT | FK | REFERENCES Patient(patient_id) |
| Appointment | doctor_id | INT | FK | REFERENCES Doctor(doctor_id) |
| Appointment | appt_date | DATE | — | NOT NULL |
| Appointment | appt_time | TIME | — | NOT NULL |
| Appointment | status | VARCHAR(20) | — | NOT NULL |
| Appointment | symptoms | TEXT | — | — |

| | | | | |
|---|---|---|---|---|
| Admission | admission_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Admission | patient_id | INT | FK | REFERENCES Patient(patient_id) |
| Admission | room_id | INT | FK | REFERENCES Room(room_id) |
| Admission | admission_date | DATE | — | NOT NULL |
| Admission | discharge_date | DATE | — | — |
| Treatment | treatment_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Treatment | appt_id | INT | FK | REFERENCES Appointment(appt_id) |
| Treatment | diagnosis | TEXT | — | — |
| Treatment | treatment_date | DATE | — | NOT NULL |
| Treatment | ward_id | INT | FK | REFERENCES Ward(ward_id) |
| Medical_Record | record_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Medical_Record | appointment_id | INT | FK | REFERENCES Appointment(appt_id) |
| Medical_Record | diagnosis | TEXT | — | — |
| Medical_Record | treatment | TEXT | — | — |
| Medical_Record | record_date | DATE | — | NOT NULL |
| Prescription | prescription_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Prescription | record_id | INT | FK | REFERENCES Medical_Record(record_id) |
| Prescription | prescription_date | DATE | — | NOT NULL |
| Medicine | med_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Medicine | med_name | VARCHAR(100) | — | NOT NULL |
| Medicine | description | TEXT | — | — |
| Medicine | price | DECIMAL(10,2) | — | NOT NULL |
| Medicine | stock_quantity | INT | — | NOT NULL |
| Prescription_Med | prescription_id | INT | PK, FK | REFERENCES Prescription(prescription_id) |
| Prescription_Med | med_id | INT | PK, FK | REFERENCES Medicine(med_id) |
| Prescription_Med | dosage | VARCHAR(50) | — | NOT NULL |
| Prescription_Med | frequency | VARCHAR(50) | — | NOT NULL |

| | | | | |
|---|---|---|---|---|
| Prescription_Med | duration | VARCHAR(50) | — | NOT NULL |
| Prescription_Med | quantity | INT | — | NOT NULL |
| Bill | bill_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Bill | patient_id | INT | FK | REFERENCES Patient(patient_id) |
| Bill | treatment_id | INT | FK | REFERENCES Treatment(treatment_id) |
| Bill | consultation_fee | DECIMAL(10,2) | — | NOT NULL |
| Bill | medicine_charges | DECIMAL(10,2) | — | NOT NULL |
| Bill | bed_charges | DECIMAL(10,2) | — | NOT NULL |
| Bill | payment_status | VARCHAR(20) | — | NOT NULL |
| Bill | bill_date | DATE | — | NOT NULL |
| Payment | payment_id | INT | PK | NOT NULL, AUTO_INCREMENT |
| Payment | bill_id | INT | FK | REFERENCES Bill(bill_id) |
| Payment | payment_date | DATE | — | NOT NULL |
| Payment | amount | DECIMAL(10,2) | — | NOT NULL |
| Payment | payment_method | VARCHAR(30) | — | NOT NULL |

**Schema Diagram**

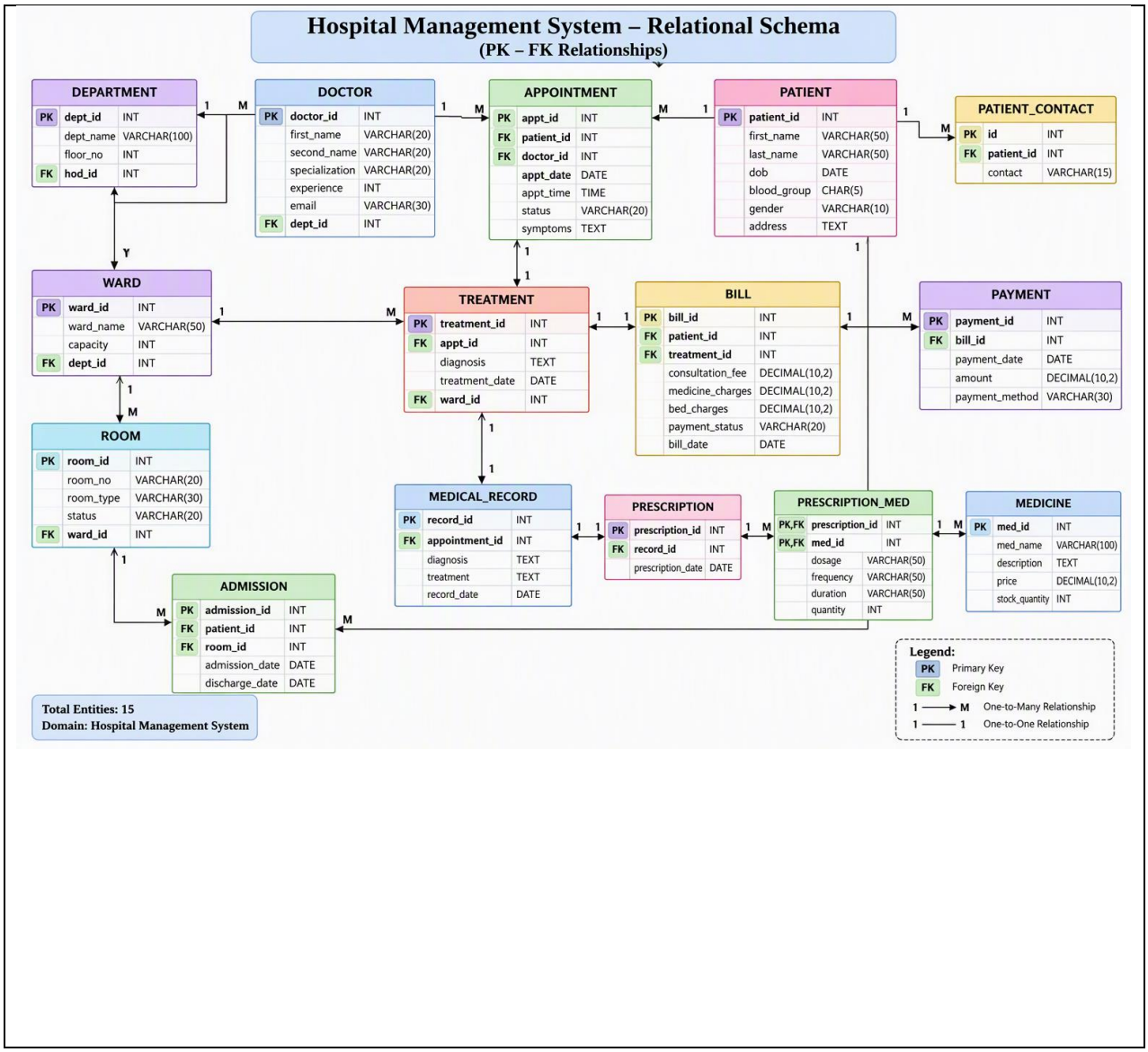



**Fig. 3.2: Relational Schema Diagram**

## 3.6 Primary Keys and Foreign Keys

**Table 3.5: Primary Key and Foreign Key Definitions**

| Table | Primary Key | Foreign Key | References (Table.Column) |
|---|---|---|---|
| Department | Department_id | HOD_id | Doctor |
| Doctor | Doctor_id | Department_id | Department |
| Appiontment | Appiontment_id | Patient_id | Patient |
| Appiontment | Appiontment_id | Doctor_id | Doctor |
| Patient | Patient_id | --------- | --------- |
| Patient_Contact | Id | Patient_id | Patient |

| | | | |
|---|---|---|---|
| Ward | Ward_id | Department_id | Department |
| Treatment | Treatment_id | Appiontement_id | Appiontment |
| Treatment | Treatment_id | Ward_id | Ward |
| Bill | Bill_id | Patient_id | Patient |
| Bill | Bill_id | Treatment_id | Treatment |
| Payment | Payment_id | Bill_id | Bill |
| Room | Room_id | Ward_id | Ward |
| Admission | Admission_id | Patient_id | Room |
| Admission | Admission_id | Room_id | Room |
| Medical_Record | Record_id | Appiontment_id | Appiontment |
| Prescription | Prescription_id | Record_id | Medical_Record |
| Prescription_Med | Prescription_id | Med_id | Prescription |
| Prescription_Med | Med_id | Prescription_id | Prescription |
| Medicine | Med_id | -------- | -------- |

## 3.7 Normalization up to 3NF

*Show the normalization process. Start from the unnormalized relation, state the functional dependencies, and show how the design reaches 1NF, 2NF and 3NF. Explain which redundancy or update anomaly each step removes.*

**UNF → 1NF → 2NF → 3NF**

**Table 3.6: Normalization Steps**

| Normal Form | Condition Satisfied | Anomaly Removed | Resulting Tables |
|---|---|---|---|
| UNF | Repeating groups present | None | Single Hospital_Record relation |
| 1NF | Atomic attributes, no repeating groups | Eliminates repeating data and nested values | Patient, Doctor, Appointment, Medicine, Prescription_Med |
| 2NF | No partial dependency on composite keys | Removes update anomalies caused by partial dependencies | Medicine, Prescription_Med and other entity-specific tables |
| 3NF | No transitive dependency | Eliminates insertion, deletion, and update anomalies | Department, Doctor, Patient, Ward, Room, Appointment, Treatment, Admission, Medical_Record, Prescription, Medicine, Prescription_Med, Bill, Payment, Patient_Contact |

# CHAPTER 4: DATABASE IMPLEMENTATION

## 4.1 Database and Table Creation – DDL

```
create database hms;
use hms;
CREATE TABLE Department(
        dept_id int primary key auto_increment,
   dept_name varchar(100) not null unique,
   floor_no int,
   hod_id int
);

create table doctor(
        DOCTOR_ID int primary key auto_increment,
        first_name varchar(20) not null,
   second_name varchar(20) not null,
   specialization varchar(20),
   Experience int check(experience >=0),
   email varchar(30) unique not null,
   dept_id int,
   foreign key (DEPT_ID) references DEPARTMENT(DEPT_ID)
);

CREATE TABLE Patient (
 patient_id INT PRIMARY KEY AUTO_INCREMENT,
 first_name VARCHAR(50) NOT NULL,
 last_name VARCHAR(50) NOT NULL,
 dob DATE,
 blood_group CHAR(5),
 gender VARCHAR(10),
 address TEXT
);

CREATE TABLE Patient_Contact (
 id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 contact VARCHAR(15),
 FOREIGN KEY (patient_id) REFERENCES Patient(patient_id)
);

CREATE TABLE Ward (
 ward_id INT PRIMARY KEY AUTO_INCREMENT,
 ward_name VARCHAR(50),
 capacity INT CHECK (capacity > 0),
 dept_id INT,
 FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Appointment (
 appt_id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 doctor_id INT NOT NULL,
 appt_date DATE NOT NULL,
 appt_time TIME NOT NULL,
 status VARCHAR(20) DEFAULT "Scheduled",
 symptoms TEXT,
 FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
 FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)
);

CREATE TABLE Treatment (
 treatment_id INT PRIMARY KEY AUTO_INCREMENT,
 appt_id INT UNIQUE NOT NULL,
 diagnosis TEXT,
 treatment_date DATE,
 ward_id INT,
 FOREIGN KEY (appt_id) REFERENCES Appointment(appt_id),
 FOREIGN KEY (ward_id) REFERENCES Ward(ward_id)
```

```
);

CREATE TABLE Bill (
 bill_id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 treatment_id INT UNIQUE,
 consultation_fee DECIMAL(10,2) DEFAULT 0,
 medicine_charges DECIMAL(10,2) DEFAULT 0,
 bed_charges DECIMAL(10,2) DEFAULT 0,
 payment_status VARCHAR(20) DEFAULT "Pending",
 bill_date DATE,
 FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
 FOREIGN KEY (treatment_id) REFERENCES Treatment(treatment_id)
);

CREATE TABLE Room (
    room_id INT PRIMARY KEY AUTO_INCREMENT,
    room_no VARCHAR(20) NOT NULL UNIQUE,
    room_type VARCHAR(30),
    status VARCHAR(20) DEFAULT 'Available',
    ward_id INT NOT NULL,
    FOREIGN KEY (ward_id) REFERENCES Ward(ward_id)
);

CREATE TABLE Admission (
    admission_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id INT NOT NULL,
    room_id INT NOT NULL,
    admission_date DATE NOT NULL,
    discharge_date DATE,
    FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
    FOREIGN KEY (room_id) REFERENCES Room(room_id)
);

CREATE TABLE Medical_Record (
    record_id INT PRIMARY KEY AUTO_INCREMENT,
    appointment_id INT NOT NULL UNIQUE,
    diagnosis TEXT,
    treatment TEXT,
    record_date DATE,
    FOREIGN KEY (appointment_id) REFERENCES Appointment(appt_id)
);

CREATE TABLE Prescription (
    prescription_id INT PRIMARY KEY AUTO_INCREMENT,
    record_id INT NOT NULL,
    prescription_date DATE NOT NULL,
    FOREIGN KEY (record_id) REFERENCES Medical_Record(record_id)
);

CREATE TABLE Medicine (
    med_id INT PRIMARY KEY AUTO_INCREMENT,
    med_name VARCHAR(100) NOT NULL UNIQUE,
    description TEXT,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT DEFAULT 0 CHECK (stock_quantity >= 0)
);

CREATE TABLE Prescription_Med (
    prescription_id INT NOT NULL,
    med_id INT NOT NULL,
    dosage VARCHAR(50),
    frequency VARCHAR(50),
    duration VARCHAR(50),
    quantity INT CHECK (quantity > 0),
    PRIMARY KEY (prescription_id, med_id),
    FOREIGN KEY (prescription_id)
       REFERENCES Prescription(prescription_id),
    FOREIGN KEY (med_id)
       REFERENCES Medicine(med_id)
);
```

```
CREATE TABLE Payment (
    payment_id INT PRIMARY KEY AUTO_INCREMENT,
    bill_id INT NOT NULL,
    payment_date DATE NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_method VARCHAR(30),
    FOREIGN KEY (bill_id) REFERENCES Bill(bill_id)
);

ALTER TABLE Department
ADD CONSTRAINT fk_department_hod
FOREIGN KEY (hod_id) REFERENCES Doctor(doctor_id);
```

**Table: admission**

| Columns: | |
|---|---|
| admission_id | int AI PK |
| patient_id | int |
| room_id | int |
| admission_date | date |
| discharge_date | date |

**Table: appointment**

| Columns: | |
|---|---|
| appt_id | int AI PK |
| patient_id | int |
| doctor_id | int |
| appt_date | date |
| appt_time | time |
| status | varchar(20) |
| symptoms | text |

**Table: bill**

| Columns: | |
|---|---|
| bill_id | int AI PK |
| patient_id | int |
| treatment_id | int |
| consultation_fee | decimal(10,2) |
| medicine_charges | decimal(10,2) |
| bed_charges | decimal(10,2) |
| payment_status | varchar(20) |
| bill_date | date |

**Table: department**

| Columns: | |
|---|---|
| dept_id | int AI PK |
| dept_name | varchar(100) |
| floor_no | int |
| hod_id | int |

**Table: doctor**

| Columns: | |
|---|---|
| DOCTOR_ID | int AI PK |
| first_name | varchar(20) |
| second_name | varchar(20) |
| specialization | varchar(20) |
| Experience | int |
| email | varchar(30) |
| dept_id | int |

**Table: medical_record**

| Columns: | |
|---|---|
| record_id | int AI PK |
| appointment_id | int |
| diagnosis | text |
| treatment | text |
| record_date | date |

**Table: medicine**

| Columns: | |
|---|---|
| med_id | int AI PK |
| med_name | varchar(100) |
| description | text |
| price | decimal(10,2) |
| stock_quantity | int |

**Table: patient**

| Columns: | |
|---|---|
| patient_id | int AI PK |
| first_name | varchar(50) |
| last_name | varchar(50) |
| dob | date |
| blood_group | char(5) |
| gender | varchar(10) |
| address | text |

**Table: patient_contact**

| Columns: | |
|---|---|
| id | int AI PK |
| patient_id | int |
| contact | varchar(15) |

**Table: payment**

| Columns: | |
|---|---|
| payment_id | int AI PK |
| bill_id | int |
| payment_date | date |
| amount | decimal(10,2) |
| payment_method | varchar(30) |

**Table: prescription**

| Columns: | |
|---|---|
| prescription_id | int AI PK |
| record_id | int |
| prescription_date | date |

**Table: prescription_med**

| Columns: | |
|---|---|
| prescription_id | int PK |
| med_id | int PK |
| dosage | varchar(50) |
| frequency | varchar(50) |
| duration | varchar(50) |
| quantity | int |

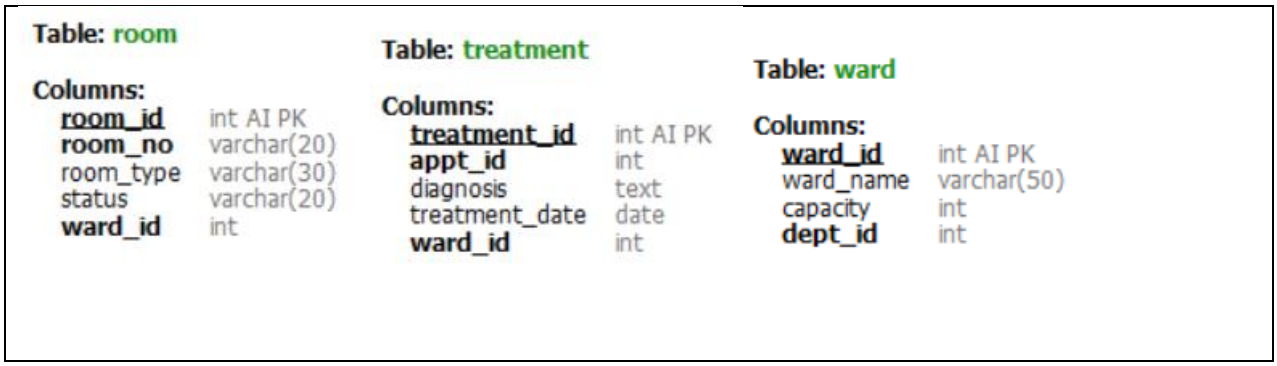
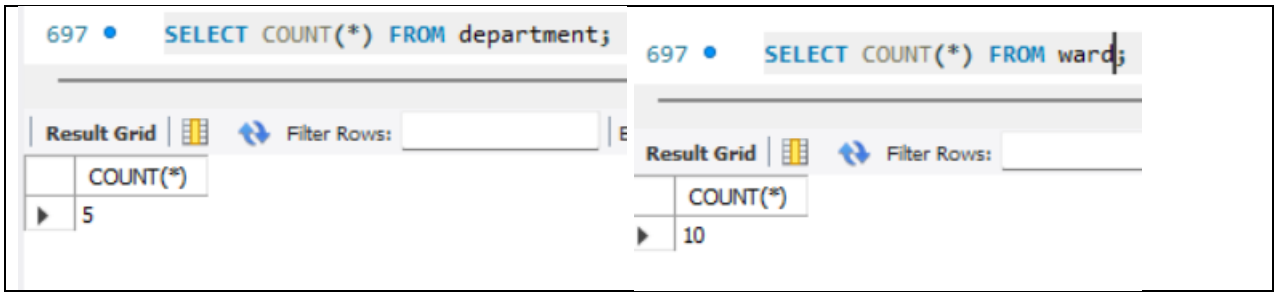



**Fig. 4.1: Successful Creation of Tables**

## 4.2 Sample Data Insertion – DML

**Table 4.1: Sample Data Summary**

| S.No | Table Name | No. of Records Inserted | Screenshot Reference |
|---|---|---|---|
| 1 | Department | 5 | 1 |
| 2 | Ward | 10 | 2 |
| 3 | Patient | 50 | 3 |
| 4 | Patient_contact | 50 | 4 |
| 5 | Medicine | 20 | 5 |
| 6 | Prescription | 50 | 6 |
| 7 | Prescription_med | 50 | 7 |
| 8 | Room | 200 | 8 |
| 9 | Treatment | 50 | 9 |
| 10 | Payment | 33 | 10 |
| 11 | Bill | 200 | 11 |
| 12 | Doctor | 10 | 12 |
| 13 | Admission | 50 | 13 |
| 14 | Appointment | 50 | 14 |
| 15 | Medical_record | 50 | 15 |



```
697 •  SELECT COUNT(*) FROM admission;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM appointment;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM bill;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM doctor;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 10 |

```
697 •  SELECT COUNT(*) FROM Medical_record;
```

Result Grid | Filter Rows: | Export:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM Medicine;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 20 |

```
697 •  SELECT COUNT(*) FROM Patient;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM Patient_contact;
```

Result Grid | Filter Rows: | Export:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM payment;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 33 |

```
697 •  SELECT COUNT(*) FROM prescription;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM prescription_med;
```

Result Grid | Filter Rows: | Export:

| COUNT(*) |
|---|
| 50 |

```
697 •  SELECT COUNT(*) FROM room;
```

Result Grid | Filter Rows:

| COUNT(*) |
|---|
| 200 |

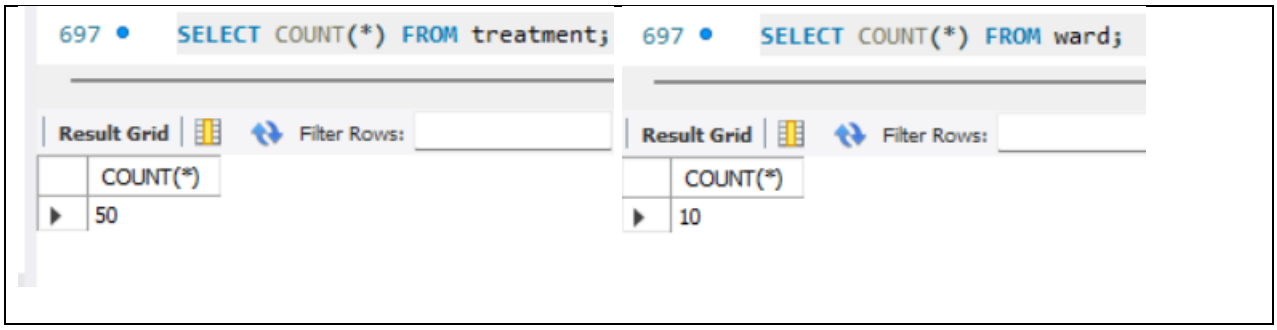



**Fig. 4.2: Sample Data Inserted into the Tables**

## 4.3 Constraints and Data Integrity

**Table 4.2: Constraints and Data Integrity Rules**

| Constraint / Validation | Applied On (Table.Column) | Purpose | Evidence / Screenshot Ref. |
|---|---|---|---|
| PRIMARY KEY | Student.StudentID | Ensures each student record is uniquely identified. | Fig. 4.2.1 |
| NOT NULL | Student.StudentName | Prevents student name from being left empty. | Fig. 4.2.2 |
| UNIQUE | Student.Email | Prevents duplicate email addresses. | Fig. 4.2.3 |
| FOREIGN KEY | Enrollment.StudentID | Maintains referential integrity between Enrollment and Student tables. | Fig. 4.2.4 |
| FOREIGN KEY | Enrollment.CourseID | Ensures enrollment references a valid course. | Fig. 4.2.5 |
| CHECK (Age >= 18) | Student.Age | Ensures only valid age values are entered. | Fig. 4.2.6 |
| DEFAULT CURRENT_DATE | Enrollment.EnrollmentDate | Automatically records the enrollment date. | Fig. 4.2.7 |
| ENUM ('Active','Inactive') | Student.Status | Restricts status values to predefined options. | Fig. 4.2.8 |

## 4.4 Data Validation Rules

Duplicate IDs Prevention

Duplicate IDs are prevented using the PRIMARY KEY constraint. A primary key ensures that each record has a unique identifier and that no two rows can contain the same ID value. If a user attempts to insert a duplicate ID, MySQL rejects the operation and returns an error.

Invalid Values Prevention

Invalid values are prevented through constraints such as NOT NULL, CHECK, UNIQUE, and data type restrictions. These constraints ensure that mandatory fields are not left empty, values fall within acceptable ranges, and data entered conforms to the specified format.

Orphan Records Prevention

Orphan records are prevented using FOREIGN KEY constraints. A foreign key requires that a value in a child table already exists in the corresponding parent table. This maintains referential integrity between related tables and prevents records from referencing non-existent entities.

**Fig. 4.3: Referential Integrity Enforced by the Database**

# CHAPTER 5: SQL OPERATIONS AND QUERIES

## 5.1 DQL – Basic Retrieval Queries

**Table 5.1: Basic Retrieval Queries**

| Q.No | Purpose of the Query | SQL Statement | Output Ref. |
|---|---|---|---|
| Q1 | Display all records from Student table | SELECT * FROM Student; | Fig. 5.1 |
| Q2 | Display selected columns from Student table | SELECT StudentID, StudentName FROM Student; | Fig. 5.2 |
| Q3 | Retrieve records matching a condition | SELECT * FROM Student WHERE Department='CSE'; | Fig. 5.3 |
| Q4 | Retrieve records using multiple conditions | SELECT * FROM Student WHERE Department='CSE' AND Year=3; | Fig. 5.4 |
| Q5 | Sort records in ascending order | SELECT * FROM Student ORDER BY StudentName ASC; | Fig. 5.5 |
| Q6 | Sort records in descending order | SELECT * FROM Student ORDER BY StudentName DESC; | Fig. 5.6 |
| Q7 | Count total records | SELECT COUNT(*) AS TotalStudents FROM Student; | Fig. 5.7 |
| Q8 | Calculate average value | SELECT AVG(Marks) AS AverageMarks FROM Student; | Fig. 5.8 |
| Q9 | Group records by category | SELECT Department, COUNT(*) FROM Student GROUP BY Department; | Fig. 5.9 |
| Q10 | Join Student and Enrollment tables | SELECT * FROM Student s INNER JOIN Enrollment e ON s.StudentID=e.StudentID; | Fig. 5.10 |
| Q11 | Display all students including those without enrollments | SELECT * FROM Student s LEFT JOIN Enrollment e ON s.StudentID=e.StudentID; | Fig. 5.11 |
| Q12 | Join multiple tables | SELECT s.StudentName, c.CourseName FROM Student s INNER JOIN Enrollment e ON s.StudentID=e.StudentID INNER JOIN Course c ON e.CourseID=c.CourseID; | Fig. 5.12 |
| Q13 | Use a subquery to find specific records | SELECT * FROM Student WHERE StudentID IN (SELECT StudentID FROM Enrollment); | Fig. 5.13 |
| Q14 | Find record with maximum value | SELECT * FROM Student WHERE Marks=(SELECT MAX(Marks) FROM Student); | Fig. 5.14 |

| | | | |
|---|---|---|---|
| Q15 | Use HAVING clause with GROUP BY | SELECT Department, COUNT(*) FROM Student GROUP BY Department HAVING COUNT(*) > 5; | Fig. 5.15 |

## 5.2 Join Operations

*Include natural join, equi join, inner join, left outer, right outer and full outer join.*

**Table 5.2: Join Operations**

| Q.No | Type of Join | SQL Statement | Output Ref. |
|---|---|---|---|
| Q10 | INNER JOIN | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s INNER JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.10 |
| Q11 | LEFT JOIN | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s LEFT JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.11 |
| Q12 | RIGHT JOIN | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s RIGHT JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.12 |
| Q13 | FULL OUTER JOIN* | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s LEFT JOIN Enrollment e ON s.StudentID = e.StudentID UNION SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s RIGHT JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.13 |
| Q14 | SELF JOIN | SELECT A.StudentName, B.StudentName AS Mentor FROM Student A JOIN Student B ON A.MentorID = B.StudentID; | Fig. 5.14 |
| Q15 | Multiple Table JOIN | SELECT s.StudentName, c.CourseName FROM Student s INNER JOIN Enrollment e ON s.StudentID = e.StudentID INNER JOIN Course c ON e.CourseID = c.CourseID; | Fig. 5.15 |

## 5.3 Aggregate Functions, GROUP BY and HAVING

| Q.No | Function Used | SQL Statement | Output Ref. |
|---|---|---|---|
| Q16 | COUNT() | SELECT COUNT(*) AS TotalStudents FROM Student; | Fig. 5.16 |
| Q17 | SUM() | SELECT SUM(FeeAmount) AS TotalFees FROM Student; | Fig. 5.17 |
| Q18 | AVG() | SELECT AVG(Marks) AS AverageMarks FROM Student; | Fig. 5.18 |
| Q19 | MIN() | SELECT MIN(Marks) AS LowestMarks FROM Student; | Fig. 5.19 |
| Q20 | MAX() | SELECT MAX(Marks) AS HighestMarks FROM Student; | Fig. 5.20 |
| Q21 | GROUP BY | SELECT Department, COUNT(*) AS TotalStudents FROM Student GROUP BY Department; | Fig. 5.21 |
| Q22 | GROUP BY with SUM() | SELECT Department, SUM(FeeAmount) AS TotalFees FROM Student GROUP BY Department; | Fig. 5.22 |

| | | | |
|---|---|---|---|
| Q23 | GROUP BY with AVG() | SELECT Department, AVG(Marks) AS AverageMarks FROM Student GROUP BY Department; | Fig. 5.23 |
| Q24 | HAVING | SELECT Department, COUNT(*) AS TotalStudents FROM Student GROUP BY Department HAVING COUNT(*) > 5; | Fig. 5.24 |
| Q25 | HAVING with AVG() | SELECT Department, AVG(Marks) AS AverageMarks FROM Student GROUP BY Department HAVING AVG(Marks) > 75; | Fig. 5.25 |

## 5.4 Nested and Correlated Subqueries

**Table 5.4: Subqueries**

| Q.No | Type of Join | SQL Statement | Output Ref. |
|---|---|---|---|
| Q10 | INNER JOIN | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s INNER JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.10 |
| Q11 | LEFT JOIN | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s LEFT JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.11 |
| Q12 | RIGHT JOIN | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s RIGHT JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.12 |
| Q13 | FULL OUTER JOIN* | SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s LEFT JOIN Enrollment e ON s.StudentID = e.StudentID UNION SELECT s.StudentID, s.StudentName, e.CourseID FROM Student s RIGHT JOIN Enrollment e ON s.StudentID = e.StudentID; | Fig. 5.13 |
| Q14 | SELF JOIN | SELECT A.StudentName, B.StudentName AS Mentor FROM Student A JOIN Student B ON A.MentorID = B.StudentID; | Fig. 5.14 |
| Q15 | Multiple Table JOIN | SELECT s.StudentName, c.CourseName FROM Student s INNER JOIN Enrollment e ON s.StudentID = e.StudentID INNER JOIN Course c ON e.CourseID = c.CourseID; | Fig. 5.15 |

## 5.5 Views and Relational Set Operations.

**Table 5.5: Views and Set Operations**

| Q.No | View / Set Operation | SQL Statement | Output Ref. |
|---|---|---|---|
| Q36 | CREATE VIEW | CREATE VIEW StudentDetails AS SELECT StudentID, StudentName, Department FROM Student; | Fig. 5.36 |
| Q37 | Querying a View | SELECT * FROM StudentDetails; | Fig. 5.37 |

| | | | |
|---|---|---|---|
| Q38 | View with JOIN | CREATE VIEW EnrollmentDetails AS SELECT s.StudentName, c.CourseName FROM Student s INNER JOIN Enrollment e ON s.StudentID = e.StudentID INNER JOIN Course c ON e.CourseID = c.CourseID; | Fig. 5.38 |
| Q39 | Querying Joined View | SELECT * FROM EnrollmentDetails; | Fig. 5.39 |
| Q40 | UNION | SELECT StudentName FROM Student UNION SELECT InstructorName FROM Instructor; | Fig. 5.40 |
| Q41 | UNION ALL | SELECT StudentName FROM Student UNION ALL SELECT InstructorName FROM Instructor; | Fig. 5.41 |
| Q42 | INTERSECT* | SELECT StudentID FROM Student WHERE Department='CSE' INTERSECT SELECT StudentID FROM Enrollment; | Fig. 5.42 |
| Q43 | EXCEPT / MINUS* | SELECT StudentID FROM Student EXCEPT SELECT StudentID FROM Enrollment; | Fig. 5.43 |

## 5.6 DCL and TCL Operations

**Table 5.6: DCL and TCL Operations**

| Q.No | View / Set Operation | SQL Statement | Output Ref. |
|---|---|---|---|
| Q36 | CREATE VIEW | CREATE VIEW StudentDetails AS SELECT StudentID, StudentName, Department FROM Student; | Fig. 5.36 |
| Q37 | Querying a View | SELECT * FROM StudentDetails; | Fig. 5.37 |
| Q38 | View with JOIN | CREATE VIEW EnrollmentDetails AS SELECT s.StudentName, c.CourseName FROM Student s INNER JOIN Enrollment e ON s.StudentID = e.StudentID INNER JOIN Course c ON e.CourseID = c.CourseID; | Fig. 5.38 |
| Q39 | Querying Joined View | SELECT * FROM EnrollmentDetails; | Fig. 5.39 |
| Q40 | UNION | SELECT StudentName FROM Student UNION SELECT InstructorName FROM Instructor; | Fig. 5.40 |
| Q41 | UNION ALL | SELECT StudentName FROM Student UNION ALL SELECT InstructorName FROM Instructor; | Fig. 5.41 |
| Q42 | INTERSECT* | SELECT StudentID FROM Student WHERE Department='CSE' INTERSECT SELECT StudentID FROM Enrollment; | Fig. 5.42 |

| | | | |
|---|---|---|---|
| Q43 | EXCEPT / MINUS* | SELECT StudentID FROM Student EXCEPT SELECT StudentID FROM Enrollment; | Fig. 5.43 |

# CHAPTER 6: MODULES AND FUNCTIONALITIES

**Table 6.1: Modules and Their Functionalities**

| S.No | Module Name | Functionality | Tables Involved |
|---|---|---|---|
| 1 | User Management | Manage user registration, login, and profile information | Users |
| 2 | Customer Management | Add, update, view, and delete customer records | Customers |
| 3 | Product Management | Manage product details and inventory information | Products |
| 4 | Order Management | Create and process customer orders | Orders, Customers, Products |
| 5 | Transaction Management | Record and track payments or transactions | Transactions, Orders |
| 6 | Report Generation | Generate summary and analytical reports | Multiple Tables |
| 7 | Search Module | Search records based on specified criteria | All Relevant Tables |
| 8 | Administration Module | Manage system settings and user permissions | Users, Roles |

# CHAPTER 7: RESULTS – SAMPLE OUTPUTS AND SCREENSHOTS

```
690 •  SELECT * FROM Patient;
691 •  SELECT first_name, second_name, specialization, Experience
692    FROM Doctor WHERE Experience > 7;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| first_name | second_name | specialization | Experience |
|---|---|---|---|
| Palgun | Reddy | Cardiology | 8 |
| Ravi Teja | Sharma | Neurology | 10 |
| Ashok | Reddy | Orthopedics | 9 |
| Sriram | Kumar | Orthopedics | 12 |
| Manikanta | Rao | Pediatrics | 8 |
| Shaurya | Kumar | General Medicine | 11 |

**Fig. 7.1: Doctor whose experience greater than 7 years**

```
3
4 •  SELECT med_name, price FROM Medicine
5    WHERE price > 30 ORDER BY price DESC;
6
7 •  SELECT COUNT(*) FROM ward;
8
```

sult Grid | Filter Rows: | Export:

| med_name | price |
|---|---|
| Azithromycin | 80.00 |
| Cough Syrup | 60.00 |
| Amoxicillin | 50.00 |
| Multivitamin | 50.00 |
| Atorvastatin | 45.00 |
| Amlodipine | 40.00 |
| Losartan | 40.00 |
| Metformin | 35.00 |
| Montelukast | 35.00 |

**Fig. 7.2: Medicines price greater than 30**

```
699 •   SELECT AVG(Experience) AS average_experience FROM Doctor;
700
701 •   select count(*) from treatment;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| average_experience |
|---|
| 8.3000 |

**Fig. 7.3: Average Experience**

```
701 •   select count(*) from treatment;
702 •   select * from treatment;
```

Result Grid | Filter Rows: | Edit: | Export/I

| treatment_id | appt_id | diagnosis | treatment_date | ward_id |
|---|---|---|---|---|
| 1 | 1 | Hypertension | 2026-09-01 | 1 |
| 2 | 2 | Migraine | 2026-09-01 | 3 |
| 3 | 3 | Fracture | 2026-09-02 | 5 |
| 4 | 4 | Common Cold | 2026-09-02 | 7 |
| 5 | 5 | Fever | 2026-09-03 | 9 |
| 6 | 6 | Chest Pain | 2026-09-03 | 1 |
| 7 | 7 | Back Pain | 2026-09-04 | 5 |
| 8 | 8 | Asthma | 2026-09-04 | 9 |
| 9 | 9 | Diabetes | 2026-09-05 | 9 |
| 10 | 10 | Headache | 2026-09-05 | 3 |
| 11 | 11 | Joint Pain | 2026-09-06 | 5 |

**Fig. 7.4: List of all the treatments**

# CHAPTER 8: TESTING AND DATA VALIDATION

**Table 8.1: Test Cases and Results**

| TC No. | Test Case Description | Input / Action | Expected Result | Actual Result | Status |
|---|---|---|---|---|---|
| TC01 | Add a new department | Insert a department record with unique department name | Department record should be inserted successfully | Department added successfully | Pass |
| TC02 | Add a doctor with valid department ID | Insert doctor record linked to an existing department | Doctor record should be inserted successfully | Doctor added successfully | Pass |
| TC03 | Add a doctor with invalid department ID | Insert doctor record with non-existing dept_id | Foreign key constraint error should occur | Error generated as expected | Pass |
| TC04 | Register a new patient | Insert patient details into Patient table | Patient record should be created successfully | Patient registered successfully | Pass |
| TC05 | Add patient contact information | Insert contact number for existing patient | Contact details should be stored successfully | Contact added successfully | Pass |
| TC06 | Schedule an appointment | Insert appointment with valid patient_id and doctor_id | Appointment should be scheduled successfully | Appointment created successfully | Pass |
| TC07 | Schedule appointment with invalid patient ID | Insert appointment using non-existing patient_id | Foreign key constraint error should occur | Error generated as expected | Pass |
| TC08 | Create treatment record | Insert treatment linked to an existing appointment | Treatment record should be created successfully | Treatment added successfully | Pass |
| TC09 | Generate medical record | Insert medical record linked to appointment | Medical record should be stored successfully | Medical record created successfully | Pass |
| TC10 | Create prescription | Insert prescription for an existing medical record | Prescription should be generated successfully | Prescription created successfully | Pass |
| TC11 | Add medicine | Insert medicine with | Medicine record | Medicine added | Pass |

| | | | | | |
|---|---|---|---|---|---|
| | details | valid price and stock quantity | should be added successfully | successfully | |
| TC12 | Add medicine to prescription | Insert record into Prescription_Med table | Medicine should be linked to prescription | Record inserted successfully | Pass |
| TC13 | Generate patient bill | Insert bill linked to treatment and patient | Bill should be generated successfully | Bill created successfully | Pass |
| TC14 | Record payment | Insert payment against an existing bill | Payment record should be stored successfully | Payment recorded successfully | Pass |
| TC15 | Retrieve all patients | Execute SELECT * FROM Patient; | List of all patients should be displayed | Records displayed correctly | Pass |
| TC16 | Retrieve doctors with experience greater than 7 years | Execute query on Doctor table | Matching doctor records should be displayed | Expected records retrieved | Pass |
| TC17 | Calculate total number of patients | Execute SELECT COUNT(*) FROM Patient; | Total patient count should be returned | Count returned successfully | Pass |
| TC18 | Calculate average doctor experience | Execute SELECT AVG(Experience) FROM Doctor; | Average experience should be displayed | Result displayed correctly | Pass |
| TC19 | Insert duplicate doctor email | Insert doctor with an existing email address | Unique constraint violation should occur | Error generated as expected | Pass |
| TC20 | Insert medicine with negative stock quantity | Insert stock quantity less than zero | CHECK constraint error should occur | Error generated as expected | Pass |

# CHAPTER 9: GITHUB REPOSITORY AND TEAM CONTRIBUTION

## 9.1 Repository Details

GitHub Repository Link: https://github.com/palgun9999/Hospital-Management-System

*The final SQL script and all project files must be pushed to the team repository.*

## 9.2 Team Responsibilities and Contribution

**Table 9.1: Team Responsibilities**

| S.No | Student Name | Roll Number | Role / Responsibility | Modules Contributed |
|---|---|---|---|---|
| 1 | D.R.PALGUN | 25B11AI241 | Data Base Administrator | MySQL Workbench, Database Implementation, Query Execution |
| 2 | K.Ashok | 25B11AI478 | Relational Model Desinger | ER Diagram Design using Draw.io, Relationship Mapping |
| 3 | P.V.S.ROHITH | 25B11AI892 | Data Base Designer | Database Schema Design, Table Structure, Constraints Definition |
| 4 | G.M.SRIRAM | 25B11AI311 | SQL Developer & Testing Engineer | SQL Queries, Data Insertion, Testing and Validation of Database Operations |

## 9.3 Review-wise Progress

**Table 9.2: Review-wise Progress Record**

| Review | Date | Deliverables Presented | Remarks of the Guide |
|---|---|---|---|
| Review 1 | 15-08-2026 | Problem Definition, Project Objectives, Scope of Hospital Management System, Literature Study, Requirements Gathering, ER Diagram Draft | Scope and objectives approved. Suggested refinement of entity relationships and normalization. |

| | | | |
|---|---|---|---|
| Review 2 | 05-09-2026 | Final ER Diagram, Relational Schema, Normalization (1NF–3NF), Table Creation Scripts, Sample Data Insertion, SQL Queries and Testing Results | Database design found satisfactory. Recommended adding more test cases, constraints, and project documentation before final submission. |

# CHAPTER 10: CONCLUSION AND FUTURE ENHANCEMENTS

## 10.1 Conclusion

The Hospital Management System database was successfully designed and implemented to provide an efficient solution for managing hospital information and operations. The project resulted in a well-structured relational database comprising 15 entities and 15 tables, namely Department, Doctor, Patient, Patient_Contact, Ward, Room, Appointment, Treatment, Admission, Medical_Record, Prescription, Medicine, Prescription_Med, Bill, and Payment. These entities were connected through appropriate primary key and foreign key relationships to ensure data integrity and consistency.

The database design was normalized up to Third Normal Form (3NF). The normalization process eliminated data redundancy, removed partial and transitive dependencies, and prevented insertion, update, and deletion anomalies. This resulted in a more efficient and reliable database structure capable of maintaining accurate healthcare records.

## 10.2 Limitations of the Present Work

Although the Hospital Management System database successfully manages patients, doctors, appointments, treatments, prescriptions, admissions, billing, and payments, it does not cover all hospital operations. The current system does not support employee attendance management, staff payroll processing, recruitment activities, or human resource management. It also does not include laboratory test management, radiology and imaging reports, ambulance services, operation theatre scheduling, or hospital equipment inventory tracking. The database is designed primarily for data storage and management and does not provide a graphical user interface, online appointment booking, patient login portal, or real-time notifications through email or SMS. Insurance claim processing and integration with external healthcare systems are also not included. Additionally, the system does not support advanced reporting, business intelligence dashboards, data analytics, or AI-based medical recommendations.

## 10.3 Future Enhancements

The current Hospital Management System provides a strong foundation for managing hospital data; however, several enhancements can be implemented in future versions to improve functionality, performance, and usability. A front-end application can be developed using web technologies or desktop frameworks to provide a user-friendly interface for hospital staff, doctors, and administrators. Additional modules such as Laboratory Management, Pharmacy Management, Employee Payroll, Staff Attendance, Ambulance Services, Insurance Processing, and Operation Theatre Scheduling can be integrated to create a complete hospital information system.

To improve database automation, stored procedures, functions, and triggers can be implemented for tasks such as automatic bill generation, medicine stock updates, appointment validation, and payment status updates. Indexes can be added on frequently searched columns such as patient IDs, doctor IDs, appointment dates, and bill IDs to improve query performance and reduce retrieval time.

# REFERENCES


6. Abraham Silberschatz, Henry F. Korth and S. Sudarshan, “Database System Concepts”, 8th Edition, McGraw-Hill, ISBN: 978-1260230508.
7. Raghurama Krishnan and Johannes Gehrke, “Database Management Systems”, 4th Edition, McGraw-Hill, ISBN: 978-1260091195.
8. Ramez Elmasri and Shamkant B. Navathe, “Fundamentals of Database Systems”, 7th Edition, Pearson, ISBN: 978-0137504277.
9. _____
10. _____

# APPENDIX A – COMPLETE SQL SCRIPT

*Paste the complete, final SQL script (CREATE, INSERT and all queries) in order, or give the exact path to the script file in the GitHub repository.*

```
use hms;
CREATE TABLE Department(
        dept_id int primary key auto_increment,
   dept_name varchar(100) not null unique,
   floor_no int,
   hod_id int
);

create table doctor(
        DOCTOR_ID int primary key auto_increment,
        first_name varchar(20) not null,
   second_name varchar(20) not null,
   specialization varchar(20),
   Experience int check(experience >=0),
   email varchar(30) unique not null,
   dept_id int,
   foreign key (DEPT_ID) references DEPARTMENT(DEPT_ID)
);

CREATE TABLE Patient (
 patient_id INT PRIMARY KEY AUTO_INCREMENT,
 first_name VARCHAR(50) NOT NULL,
 last_name VARCHAR(50) NOT NULL,
 dob DATE,
 blood_group CHAR(5),
 gender VARCHAR(10),
 address TEXT
);

CREATE TABLE Patient_Contact (
 id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 contact VARCHAR(15),
 FOREIGN KEY (patient_id) REFERENCES Patient(patient_id)
);

CREATE TABLE Ward (
 ward_id INT PRIMARY KEY AUTO_INCREMENT,
 ward_name VARCHAR(50),
 capacity INT CHECK (capacity > 0),
 dept_id INT,
 FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Appointment (
 appt_id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 doctor_id INT NOT NULL,
 appt_date DATE NOT NULL,
 appt_time TIME NOT NULL,
 status VARCHAR(20) DEFAULT "Scheduled",
 symptoms TEXT,
 FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
 FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id)
);

CREATE TABLE Treatment (
```

```
 treatment_id INT PRIMARY KEY AUTO_INCREMENT,
 appt_id INT UNIQUE NOT NULL,
 diagnosis TEXT,
 treatment_date DATE,
 ward_id INT,
 FOREIGN KEY (appt_id) REFERENCES Appointment(appt_id),
 FOREIGN KEY (ward_id) REFERENCES Ward(ward_id)
);

CREATE TABLE Bill (
 bill_id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 treatment_id INT UNIQUE,
 consultation_fee DECIMAL(10,2) DEFAULT 0,
 medicine_charges DECIMAL(10,2) DEFAULT 0,
 bed_charges DECIMAL(10,2) DEFAULT 0,
 payment_status VARCHAR(20) DEFAULT "Pending",
 bill_date DATE,
 FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
 FOREIGN KEY (treatment_id) REFERENCES Treatment(treatment_id)
);

CREATE TABLE Room (
   room_id INT PRIMARY KEY AUTO_INCREMENT,
   room_no VARCHAR(20) NOT NULL UNIQUE,
   room_type VARCHAR(30),
   status VARCHAR(20) DEFAULT 'Available',
   ward_id INT NOT NULL,
   FOREIGN KEY (ward_id) REFERENCES Ward(ward_id)
);

CREATE TABLE Admission (
   admission_id INT PRIMARY KEY AUTO_INCREMENT,
   patient_id INT NOT NULL,
   room_id INT NOT NULL,
   admission_date DATE NOT NULL,
   discharge_date DATE,
   FOREIGN KEY (patient_id) REFERENCES Patient(patient_id),
   FOREIGN KEY (room_id) REFERENCES Room(room_id)
);

CREATE TABLE Medical_Record (
   record_id INT PRIMARY KEY AUTO_INCREMENT,
   appointment_id INT NOT NULL UNIQUE,
   diagnosis TEXT,
   treatment TEXT,
   record_date DATE,
   FOREIGN KEY (appointment_id) REFERENCES Appointment(appt_id)
);

CREATE TABLE Prescription (
   prescription_id INT PRIMARY KEY AUTO_INCREMENT,
   record_id INT NOT NULL,
   prescription_date DATE NOT NULL,
   FOREIGN KEY (record_id) REFERENCES Medical_Record(record_id)
);

CREATE TABLE Medicine (
   med_id INT PRIMARY KEY AUTO_INCREMENT,
   med_name VARCHAR(100) NOT NULL UNIQUE,
   description TEXT,
   price DECIMAL(10,2) NOT NULL,
   stock_quantity INT DEFAULT 0 CHECK (stock_quantity >= 0)
```

```
);

CREATE TABLE Prescription_Med (
   prescription_id INT NOT NULL,
   med_id INT NOT NULL,
   dosage VARCHAR(50),
   frequency VARCHAR(50),
   duration VARCHAR(50),
   quantity INT CHECK (quantity > 0),
   PRIMARY KEY (prescription_id, med_id),
   FOREIGN KEY (prescription_id)
      REFERENCES Prescription(prescription_id),
   FOREIGN KEY (med_id)
      REFERENCES Medicine(med_id)
);

CREATE TABLE Payment (
   payment_id INT PRIMARY KEY AUTO_INCREMENT,
   bill_id INT NOT NULL,
   payment_date DATE NOT NULL,
   amount DECIMAL(10,2) NOT NULL,
   payment_method VARCHAR(30),
   FOREIGN KEY (bill_id) REFERENCES Bill(bill_id)
);

ALTER TABLE Department
ADD CONSTRAINT fk_department_hod
FOREIGN KEY (hod_id) REFERENCES Doctor(doctor_id);

USE hms;

SET FOREIGN_KEY_CHECKS = 0;

INSERT INTO Department
(dept_id, dept_name, floor_no, hod_id)
VALUES
(1, 'Cardiology', 1, 0),
(2, 'Neurology', 1, 0),
(3, 'Orthopedics', 2, 0),
(4, 'Pediatrics', 2, 0),
(5, 'General Medicine', 3, 0);

INSERT INTO Doctor
(doctor_id, first_name, second_name, specialization, experience, email, dept_id)
VALUES
(1, 'Palgun', 'Reddy', 'Cardiology', 8, 'palgun1@hms.com', 1),
(2, 'Rohith', 'Kumar', 'Cardiology', 7, 'rohith2@hms.com', 1),

(3, 'Ravi Teja', 'Sharma', 'Neurology', 10, 'raviteja3@hms.com', 2),
(4, 'Vyas', 'Rao', 'Neurology', 6, 'vyas4@hms.com', 2),

(5, 'Ashok', 'Reddy', 'Orthopedics', 9, 'ashok5@hms.com', 3),
(6, 'Sriram', 'Kumar', 'Orthopedics', 12, 'sriram6@hms.com', 3),

(7, 'Manikanta', 'Rao', 'Pediatrics', 8, 'manikanta7@hms.com', 4),
(8, 'Joshith', 'Reddy', 'Pediatrics', 5, 'joshith8@hms.com', 4),

(9, 'Shaurya', 'Kumar', 'General Medicine', 11, 'shaurya9@hms.com', 5),
(10, 'Haravind', 'Rao', 'General Medicine', 7, 'haravind10@hms.com', 5);

UPDATE Department SET hod_id = 1 WHERE dept_id = 1;
UPDATE Department SET hod_id = 3 WHERE dept_id = 2;
UPDATE Department SET hod_id = 5 WHERE dept_id = 3;
```

```
UPDATE Department SET hod_id = 7 WHERE dept_id = 4;
UPDATE Department SET hod_id = 9 WHERE dept_id = 5;

INSERT INTO Patient
(patient_id, first_name, last_name, dob, blood_group, gender, address)
VALUES
(1, 'Arjun', 'Reddy', '2001-05-12', 'A+', 'Male', 'Proddatur'),
(2, 'Rahul', 'Kumar', '1998-08-21', 'B+', 'Male', 'Kadapa'),
(3, 'Sanjay', 'Rao', '2000-02-15', 'O+', 'Male', 'Kurnool'),
(4, 'Priya', 'Sharma', '1999-11-03', 'AB+', 'Female', 'Tirupati'),
(5, 'Ananya', 'Reddy', '2002-06-18', 'A-', 'Female', 'Nellore'),
(6, 'Kiran', 'Kumar', '1997-09-25', 'B-', 'Male', 'Anantapur'),
(7, 'Sneha', 'Rao', '2001-01-30', 'O-', 'Female', 'Kadapa'),
(8, 'Vikram', 'Sharma', '1996-04-14', 'AB-', 'Male', 'Kurnool'),
(9, 'Pooja', 'Reddy', '2003-07-09', 'A+', 'Female', 'Proddatur'),
(10, 'Nikhil', 'Kumar', '1999-12-22', 'B+', 'Male', 'Tirupati'),

(11, 'Meena', 'Rao', '2000-03-17', 'O+', 'Female', 'Nellore'),
(12, 'Aditya', 'Sharma', '1998-10-05', 'A+', 'Male', 'Kadapa'),
(13, 'Swathi', 'Reddy', '2002-08-11', 'B+', 'Female', 'Kurnool'),
(14, 'Harsha', 'Kumar', '1997-05-27', 'O+', 'Male', 'Proddatur'),
(15, 'Divya', 'Rao', '2001-09-19', 'AB+', 'Female', 'Anantapur'),
(16, 'Manoj', 'Sharma', '1995-02-08', 'A-', 'Male', 'Kadapa'),
(17, 'Keerthi', 'Reddy', '2003-04-23', 'B-', 'Female', 'Tirupati'),
(18, 'Rakesh', 'Kumar', '1998-06-29', 'O-', 'Male', 'Nellore'),
(19, 'Aishwarya', 'Rao', '2000-11-16', 'A+', 'Female', 'Kurnool'),
(20, 'Suresh', 'Sharma', '1994-01-12', 'B+', 'Male', 'Proddatur'),

(21, 'Lakshmi', 'Reddy', '1999-07-28', 'O+', 'Female', 'Kadapa'),
(22, 'Varun', 'Kumar', '2001-03-06', 'AB+', 'Male', 'Anantapur'),
(23, 'Deepika', 'Rao', '2002-12-14', 'A+', 'Female', 'Tirupati'),
(24, 'Tarun', 'Sharma', '1997-08-30', 'B+', 'Male', 'Nellore'),
(25, 'Nandini', 'Reddy', '2000-05-21', 'O-', 'Female', 'Proddatur'),
(26, 'Rohit', 'Kumar', '1996-10-17', 'A-', 'Male', 'Kadapa'),
(27, 'Bhavya', 'Rao', '2003-02-09', 'B-', 'Female', 'Kurnool'),
(28, 'Ajay', 'Sharma', '1998-09-13', 'O+', 'Male', 'Anantapur'),
(29, 'Kavya', 'Reddy', '2001-06-25', 'AB-', 'Female', 'Tirupati'),
(30, 'Pranav', 'Kumar', '1995-12-03', 'A+', 'Male', 'Nellore'),

(31, 'Sowmya', 'Rao', '2002-01-18', 'B+', 'Female', 'Proddatur'),
(32, 'Mahesh', 'Sharma', '1997-04-07', 'O+', 'Male', 'Kadapa'),
(33, 'Tejaswini', 'Reddy', '2000-08-26', 'A-', 'Female', 'Kurnool'),
(34, 'Abhishek', 'Kumar', '1999-03-15', 'B-', 'Male', 'Tirupati'),
(35, 'Pallavi', 'Rao', '2001-10-20', 'O+', 'Female', 'Nellore'),
(36, 'Surya', 'Sharma', '1996-07-11', 'AB+', 'Male', 'Proddatur'),
(37, 'Navya', 'Reddy', '2003-05-04', 'A+', 'Female', 'Kadapa'),
(38, 'Vamshi', 'Kumar', '1998-11-27', 'B+', 'Male', 'Anantapur'),
(39, 'Mounika', 'Rao', '2002-09-08', 'O-', 'Female', 'Kurnool'),
(40, 'Chaitanya', 'Sharma', '1995-06-16', 'A+', 'Male', 'Tirupati'),

(41, 'Sravani', 'Reddy', '2000-02-28', 'B+', 'Female', 'Nellore'),
(42, 'Karthik', 'Kumar', '1997-12-10', 'O+', 'Male', 'Proddatur'),
(43, 'Amrutha', 'Rao', '2001-08-19', 'AB+', 'Female', 'Kadapa'),
(44, 'Sai', 'Sharma', '1999-05-31', 'A-', 'Male', 'Kurnool'),
(45, 'Lasya', 'Reddy', '2003-03-22', 'B-', 'Female', 'Anantapur'),
(46, 'Naveen', 'Kumar', '1996-09-06', 'O+', 'Male', 'Tirupati'),
(47, 'Hema', 'Rao', '2000-12-24', 'A+', 'Female', 'Nellore'),
(48, 'Dinesh', 'Sharma', '1998-04-13', 'B+', 'Male', 'Proddatur'),
(49, 'Pavani', 'Reddy', '2002-07-17', 'O-', 'Female', 'Kadapa'),
(50, 'Yashwanth', 'Kumar', '1997-01-29', 'AB+', 'Male', 'Kurnool');

INSERT INTO Ward
(ward_id, ward_name, capacity, dept_id)
```

```
VALUES
(1, 'Cardiology Ward A', 20, 1),
(2, 'Cardiology Ward B', 20, 1),

(3, 'Neurology Ward A', 20, 2),
(4, 'Neurology Ward B', 20, 2),

(5, 'Orthopedics Ward A', 20, 3),
(6, 'Orthopedics Ward B', 20, 3),

(7, 'Pediatrics Ward A', 20, 4),
(8, 'Pediatrics Ward B', 20, 4),

(9, 'General Medicine Ward A', 20, 5),
(10, 'General Medicine Ward B', 20, 5);

INSERT INTO Room
(room_id, room_no, room_type, status, ward_id)
VALUES
-- Cardiology Ward A
(1,'101','General','Available',1),
(2,'102','General','Available',1),
(3,'103','General','Available',1),
(4,'104','General','Available',1),
(5,'105','General','Available',1),
(6,'106','General','Available',1),
(7,'107','General','Available',1),
(8,'108','General','Available',1),
(9,'109','General','Available',1),
(10,'110','General','Available',1),
(11,'111','General','Available',1),
(12,'112','General','Available',1),
(13,'113','General','Available',1),
(14,'114','General','Available',1),
(15,'115','General','Available',1),
(16,'116','General','Available',1),
(17,'117','General','Available',1),
(18,'118','General','Available',1),
(19,'119','General','Available',1),
(20,'120','General','Available',1),

-- Cardiology Ward B
(21,'121','General','Available',2),
(22,'122','General','Available',2),
(23,'123','General','Available',2),
(24,'124','General','Available',2),
(25,'125','General','Available',2),
(26,'126','General','Available',2),
(27,'127','General','Available',2),
(28,'128','General','Available',2),
(29,'129','General','Available',2),
(30,'130','General','Available',2),
(31,'131','General','Available',2),
(32,'132','General','Available',2),
(33,'133','General','Available',2),
(34,'134','General','Available',2),
(35,'135','General','Available',2),
(36,'136','General','Available',2),
(37,'137','General','Available',2),
(38,'138','General','Available',2),
(39,'139','General','Available',2),
(40,'140','General','Available',2),
```

```
-- Neurology Ward A
(41,'201','General','Available',3),
(42,'202','General','Available',3),
(43,'203','General','Available',3),
(44,'204','General','Available',3),
(45,'205','General','Available',3),
(46,'206','General','Available',3),
(47,'207','General','Available',3),
(48,'208','General','Available',3),
(49,'209','General','Available',3),
(50,'210','General','Available',3),
(51,'211','General','Available',3),
(52,'212','General','Available',3),
(53,'213','General','Available',3),
(54,'214','General','Available',3),
(55,'215','General','Available',3),
(56,'216','General','Available',3),
(57,'217','General','Available',3),
(58,'218','General','Available',3),
(59,'219','General','Available',3),
(60,'220','General','Available',3),

-- Neurology Ward B
(61,'221','General','Available',4),
(62,'222','General','Available',4),
(63,'223','General','Available',4),
(64,'224','General','Available',4),
(65,'225','General','Available',4),
(66,'226','General','Available',4),
(67,'227','General','Available',4),
(68,'228','General','Available',4),
(69,'229','General','Available',4),
(70,'230','General','Available',4),
(71,'231','General','Available',4),
(72,'232','General','Available',4),
(73,'233','General','Available',4),
(74,'234','General','Available',4),
(75,'235','General','Available',4),
(76,'236','General','Available',4),
(77,'237','General','Available',4),
(78,'238','General','Available',4),
(79,'239','General','Available',4),
(80,'240','General','Available',4),

-- Orthopedics Ward A
(81,'301','General','Available',5),
(82,'302','General','Available',5),
(83,'303','General','Available',5),
(84,'304','General','Available',5),
(85,'305','General','Available',5),
(86,'306','General','Available',5),
(87,'307','General','Available',5),
(88,'308','General','Available',5),
(89,'309','General','Available',5),
(90,'310','General','Available',5),
(91,'311','General','Available',5),
(92,'312','General','Available',5),
(93,'313','General','Available',5),
(94,'314','General','Available',5),
(95,'315','General','Available',5),
(96,'316','General','Available',5),
(97,'317','General','Available',5),
(98,'318','General','Available',5),
```

```
(99,'319','General','Available',5),
(100,'320','General','Available',5),

-- Orthopedics Ward B
(101,'321','General','Available',6),
(102,'322','General','Available',6),
(103,'323','General','Available',6),
(104,'324','General','Available',6),
(105,'325','General','Available',6),
(106,'326','General','Available',6),
(107,'327','General','Available',6),
(108,'328','General','Available',6),
(109,'329','General','Available',6),
(110,'330','General','Available',6),
(111,'331','General','Available',6),
(112,'332','General','Available',6),
(113,'333','General','Available',6),
(114,'334','General','Available',6),
(115,'335','General','Available',6),
(116,'336','General','Available',6),
(117,'337','General','Available',6),
(118,'338','General','Available',6),
(119,'339','General','Available',6),
(120,'340','General','Available',6),

-- Pediatrics Ward A
(121,'401','General','Available',7),
(122,'402','General','Available',7),
(123,'403','General','Available',7),
(124,'404','General','Available',7),
(125,'405','General','Available',7),
(126,'406','General','Available',7),
(127,'407','General','Available',7),
(128,'408','General','Available',7),
(129,'409','General','Available',7),
(130,'410','General','Available',7),
(131,'411','General','Available',7),
(132,'412','General','Available',7),
(133,'413','General','Available',7),
(134,'414','General','Available',7),
(135,'415','General','Available',7),
(136,'416','General','Available',7),
(137,'417','General','Available',7),
(138,'418','General','Available',7),
(139,'419','General','Available',7),
(140,'420','General','Available',7),

-- Pediatrics Ward B
(141,'421','General','Available',8),
(142,'422','General','Available',8),
(143,'423','General','Available',8),
(144,'424','General','Available',8),
(145,'425','General','Available',8),
(146,'426','General','Available',8),
(147,'427','General','Available',8),
(148,'428','General','Available',8),
(149,'429','General','Available',8),
(150,'430','General','Available',8),
(151,'431','General','Available',8),
(152,'432','General','Available',8),
(153,'433','General','Available',8),
(154,'434','General','Available',8),
(155,'435','General','Available',8),
```

```
(156,'436','General','Available',8),
(157,'437','General','Available',8),
(158,'438','General','Available',8),
(159,'439','General','Available',8),
(160,'440','General','Available',8),

-- General Medicine Ward A
(161,'501','General','Available',9),
(162,'502','General','Available',9),
(163,'503','General','Available',9),
(164,'504','General','Available',9),
(165,'505','General','Available',9),
(166,'506','General','Available',9),
(167,'507','General','Available',9),
(168,'508','General','Available',9),
(169,'509','General','Available',9),
(170,'510','General','Available',9),
(171,'511','General','Available',9),
(172,'512','General','Available',9),
(173,'513','General','Available',9),
(174,'514','General','Available',9),
(175,'515','General','Available',9),
(176,'516','General','Available',9),
(177,'517','General','Available',9),
(178,'518','General','Available',9),
(179,'519','General','Available',9),
(180,'520','General','Available',9),

-- General Medicine Ward B
(181,'521','General','Available',10),
(182,'522','General','Available',10),
(183,'523','General','Available',10),
(184,'524','General','Available',10),
(185,'525','General','Available',10),
(186,'526','General','Available',10),
(187,'527','General','Available',10),
(188,'528','General','Available',10),
(189,'529','General','Available',10),
(190,'530','General','Available',10),
(191,'531','General','Available',10),
(192,'532','General','Available',10),
(193,'533','General','Available',10),
(194,'534','General','Available',10),
(195,'535','General','Available',10),
(196,'536','General','Available',10),
(197,'537','General','Available',10),
(198,'538','General','Available',10),
(199,'539','General','Available',10),
(200,'540','General','Available',10);

INSERT INTO Treatment
(treatment_id, appt_id, diagnosis, treatment_date, ward_id)
VALUES
(1,1,'Hypertension','2026-09-01',1),
(2,2,'Migraine','2026-09-01',3),
(3,3,'Fracture','2026-09-02',5),
(4,4,'Common Cold','2026-09-02',7),
(5,5,'Fever','2026-09-03',9),
(6,6,'Chest Pain','2026-09-03',1),
(7,7,'Back Pain','2026-09-04',5),
(8,8,'Asthma','2026-09-04',9),
(9,9,'Diabetes','2026-09-05',9),
(10,10,'Headache','2026-09-05',3),
```

(11,11,'Joint Pain','2026-09-06',5),
(12,12,'Gastritis','2026-09-06',9),
(13,13,'Allergy','2026-09-07',7),
(14,14,'Viral Fever','2026-09-07',9),
(15,15,'Heart Disease','2026-09-08',1),
(16,16,'Migraine','2026-09-08',3),
(17,17,'Bone Injury','2026-09-09',5),
(18,18,'Skin Infection','2026-09-09',7),
(19,19,'Blood Pressure','2026-09-10',1),
(20,20,'Stomach Pain','2026-09-10',9),
(21,21,'Arthritis','2026-09-11',5),
(22,22,'Cold','2026-09-11',7),
(23,23,'Diabetes','2026-09-12',9),
(24,24,'Neurological Disorder','2026-09-12',3),
(25,25,'Chest Infection','2026-09-13',1),
(26,26,'Fever','2026-09-13',9),
(27,27,'Fracture','2026-09-14',5),
(28,28,'Allergy','2026-09-14',7),
(29,29,'Hypertension','2026-09-15',1),
(30,30,'Migraine','2026-09-15',3),
(31,31,'Joint Pain','2026-09-16',5),
(32,32,'Gastritis','2026-09-16',9),
(33,33,'Asthma','2026-09-17',9),
(34,34,'Viral Fever','2026-09-17',7),
(35,35,'Back Pain','2026-09-18',5),
(36,36,'Diabetes','2026-09-18',9),
(37,37,'Headache','2026-09-19',3),
(38,38,'Heart Disease','2026-09-19',1),
(39,39,'Bone Injury','2026-09-20',5),
(40,40,'Skin Allergy','2026-09-20',7),
(41,41,'Fever','2026-09-21',9),
(42,42,'Hypertension','2026-09-21',1),
(43,43,'Migraine','2026-09-22',3),
(44,44,'Arthritis','2026-09-22',5),
(45,45,'Common Cold','2026-09-23',7),
(46,46,'Stomach Infection','2026-09-23',9),
(47,47,'Chest Pain','2026-09-24',1),
(48,48,'Back Pain','2026-09-24',5),
(49,49,'Diabetes','2026-09-25',9),
(50,50,'Viral Infection','2026-09-25',7);

INSERT INTO Bill
(bill_id, patient_id, treatment_id, consultation_fee, medicine_charges, bed_charges, payment_status, bill_date)
VALUES
(1,1,1,500,250,1000,'Paid','2026-09-01'),
(2,2,2,500,300,1000,'Paid','2026-09-01'),
(3,3,3,700,500,1500,'Pending','2026-09-02'),
(4,4,4,400,150,800,'Paid','2026-09-02'),
(5,5,5,400,200,800,'Pending','2026-09-03'),
(6,6,6,600,350,1200,'Paid','2026-09-03'),
(7,7,7,500,250,1000,'Paid','2026-09-04'),
(8,8,8,500,300,1000,'Pending','2026-09-04'),
(9,9,9,600,400,1200,'Paid','2026-09-05'),
(10,10,10,400,200,800,'Paid','2026-09-05'),
(11,11,11,500,250,1000,'Pending','2026-09-06'),
(12,12,12,450,200,800,'Paid','2026-09-06'),
(13,13,13,400,150,800,'Paid','2026-09-07'),
(14,14,14,400,200,800,'Pending','2026-09-07'),
(15,15,15,700,500,1500,'Paid','2026-09-08'),
(16,16,16,500,300,1000,'Paid','2026-09-08'),
(17,17,17,700,450,1500,'Pending','2026-09-09'),
(18,18,18,400,250,800,'Paid','2026-09-09'),
(19,19,19,500,200,1000,'Paid','2026-09-10'),

```
(20,20,20,450,250,800,'Pending','2026-09-10'),
(21,21,21,500,300,1000,'Paid','2026-09-11'),
(22,22,22,400,150,800,'Paid','2026-09-11'),
(23,23,23,600,400,1200,'Pending','2026-09-12'),
(24,24,24,600,350,1200,'Paid','2026-09-12'),
(25,25,25,600,400,1200,'Paid','2026-09-13'),
(26,26,26,400,200,800,'Pending','2026-09-13'),
(27,27,27,700,500,1500,'Paid','2026-09-14'),
(28,28,28,400,150,800,'Paid','2026-09-14'),
(29,29,29,500,250,1000,'Pending','2026-09-15'),
(30,30,30,500,300,1000,'Paid','2026-09-15'),
(31,31,31,500,250,1000,'Paid','2026-09-16'),
(32,32,32,450,200,800,'Pending','2026-09-16'),
(33,33,33,500,300,1000,'Paid','2026-09-17'),
(34,34,34,400,200,800,'Paid','2026-09-17'),
(35,35,35,500,250,1000,'Pending','2026-09-18'),
(36,36,36,600,400,1200,'Paid','2026-09-18'),
(37,37,37,400,200,800,'Paid','2026-09-19'),
(38,38,38,700,500,1500,'Pending','2026-09-19'),
(39,39,39,700,450,1500,'Paid','2026-09-20'),
(40,40,40,400,250,800,'Paid','2026-09-20'),
(41,41,41,400,200,800,'Pending','2026-09-21'),
(42,42,42,500,250,1000,'Paid','2026-09-21'),
(43,43,43,500,300,1000,'Paid','2026-09-22'),
(44,44,44,500,300,1000,'Pending','2026-09-22'),
(45,45,45,400,150,800,'Paid','2026-09-23'),
(46,46,46,450,250,800,'Paid','2026-09-23'),
(47,47,47,600,350,1200,'Pending','2026-09-24'),
(48,48,48,500,250,1000,'Paid','2026-09-24'),
(49,49,49,600,400,1200,'Paid','2026-09-25'),
(50,50,50,400,200,800,'Pending','2026-09-25');

INSERT INTO Medical_Record
(record_id, appointment_id, diagnosis, treatment, record_date)
SELECT
   treatment_id,
   appt_id,
   diagnosis,
   CONCAT('Treatment for ', diagnosis),
   treatment_date
FROM Treatment;

INSERT INTO Prescription
(prescription_id, record_id, prescription_date)
SELECT
   record_id,
   record_id,
   record_date
FROM Medical_Record;

INSERT INTO Medicine
(med_id, med_name, description, price, stock_quantity)
VALUES
(1,'Paracetamol','Used for fever and pain',20.00,500),
(2,'Amoxicillin','Antibiotic',50.00,300),
(3,'Azithromycin','Antibiotic',80.00,250),
(4,'Ibuprofen','Pain relief medicine',30.00,400),
(5,'Cetirizine','Used for allergy',15.00,350),
(6,'Omeprazole','Used for acidity',25.00,450),
(7,'Metformin','Used for diabetes',35.00,300),
(8,'Amlodipine','Used for blood pressure',40.00,250),
(9,'Atorvastatin','Cholesterol medicine',45.00,200),
(10,'Pantoprazole','Used for gastric problems',30.00,400),
```

```
(11,'Diclofenac','Pain relief medicine',25.00,300),
(12,'Montelukast','Used for asthma',35.00,250),
(13,'Losartan','Blood pressure medicine',40.00,300),
(14,'Aspirin','Used for heart conditions',20.00,350),
(15,'Calcium','Bone supplement',30.00,400),
(16,'Vitamin D','Vitamin supplement',25.00,400),
(17,'Cough Syrup','Used for cough',60.00,200),
(18,'ORS','Used for dehydration',15.00,500),
(19,'Multivitamin','Vitamin supplement',50.00,300),
(20,'Antacid','Used for acidity',20.00,400);

INSERT INTO Prescription_Med
(prescription_id, med_id, dosage, frequency, duration, quantity)
VALUES
(1,8,'5 mg','Once daily','30 days',30),
(2,1,'500 mg','Twice daily','5 days',10),
(3,11,'50 mg','Twice daily','7 days',14),
(4,1,'500 mg','Twice daily','5 days',10),
(5,1,'500 mg','Three times daily','5 days',15),
(6,14,'75 mg','Once daily','30 days',30),
(7,4,'400 mg','Twice daily','5 days',10),
(8,12,'10 mg','Once daily','15 days',15),
(9,7,'500 mg','Twice daily','30 days',60),
(10,1,'500 mg','Twice daily','5 days',10),
(11,11,'50 mg','Twice daily','7 days',14),
(12,6,'20 mg','Once daily','15 days',15),
(13,5,'10 mg','Once daily','10 days',10),
(14,1,'500 mg','Three times daily','5 days',15),
(15,14,'75 mg','Once daily','30 days',30),
(16,2,'500 mg','Twice daily','7 days',14),
(17,11,'50 mg','Twice daily','7 days',14),
(18,5,'10 mg','Once daily','10 days',10),
(19,8,'5 mg','Once daily','30 days',30),
(20,6,'20 mg','Once daily','15 days',15),
(21,4,'400 mg','Twice daily','7 days',14),
(22,17,'10 ml','Three times daily','5 days',1),
(23,7,'500 mg','Twice daily','30 days',60),
(24,12,'10 mg','Once daily','15 days',15),
(25,3,'500 mg','Once daily','5 days',5),
(26,1,'500 mg','Three times daily','5 days',15),
(27,11,'50 mg','Twice daily','7 days',14),
(28,5,'10 mg','Once daily','10 days',10),
(29,13,'50 mg','Once daily','30 days',30),
(30,1,'500 mg','Twice daily','5 days',10),
(31,11,'50 mg','Twice daily','7 days',14),
(32,10,'40 mg','Once daily','15 days',15),
(33,12,'10 mg','Once daily','15 days',15),
(34,1,'500 mg','Three times daily','5 days',15),
(35,4,'400 mg','Twice daily','7 days',14),
(36,7,'500 mg','Twice daily','30 days',60),
(37,1,'500 mg','Twice daily','5 days',10),
(38,14,'75 mg','Once daily','30 days',30),
(39,11,'50 mg','Twice daily','7 days',14),
(40,5,'10 mg','Once daily','10 days',10),
(41,1,'500 mg','Three times daily','5 days',15),
(42,8,'5 mg','Once daily','30 days',30),
(43,1,'500 mg','Twice daily','5 days',10),
(44,4,'400 mg','Twice daily','7 days',14),
(45,17,'10 ml','Three times daily','5 days',1),
(46,6,'20 mg','Once daily','15 days',15),
(47,14,'75 mg','Once daily','30 days',30),
(48,11,'50 mg','Twice daily','7 days',14),
(49,7,'500 mg','Twice daily','30 days',60),
```

```
(50,3,'500 mg','Once daily','5 days',5);

SELECT * FROM Patient;
SELECT first_name, second_name, specialization, Experience
FROM Doctor WHERE Experience > 7;

SELECT med_name, price FROM Medicine
WHERE price > 30 ORDER BY price DESC;

SELECT COUNT(*) FROM ward;

SELECT AVG(Experience) AS average_experience FROM Doctor;

select count(*) from treatment;
select * from treatment;

SET FOREIGN_KEY_CHECKS = 0;

INSERT INTO Appointment
(appt_id, patient_id, doctor_id, appt_date, appt_time, status, symptoms)
VALUES
(1,1,1,'2026-09-01','09:00:00','Completed','Headache, dizziness'),
(2,2,3,'2026-09-01','09:30:00','Completed','Severe headache, nausea'),
(3,3,5,'2026-09-02','10:00:00','Completed','Swelling, pain in limb'),
(4,4,7,'2026-09-02','10:30:00','Completed','Runny nose, sneezing'),
(5,5,9,'2026-09-03','11:00:00','Completed','High temperature, chills'),
(6,6,2,'2026-09-03','11:30:00','Completed','Chest tightness, discomfort'),
(7,7,6,'2026-09-04','12:00:00','Completed','Lower back pain'),
(8,8,10,'2026-09-04','12:30:00','Completed','Shortness of breath, wheezing'),
(9,9,9,'2026-09-05','14:00:00','Completed','Frequent urination, fatigue'),
(10,10,4,'2026-09-05','14:30:00','Completed','Persistent headache'),
(11,11,5,'2026-09-06','15:00:00','Completed','Joint stiffness and pain'),
(12,12,10,'2026-09-06','15:30:00','Completed','Stomach pain, acidity'),
(13,13,8,'2026-09-07','16:00:00','Completed','Skin rash, itching'),
(14,14,9,'2026-09-07','16:30:00','Completed','Fever with body ache'),
(15,15,1,'2026-09-08','17:00:00','Completed','Chest pain, breathlessness'),
(16,16,3,'2026-09-08','17:30:00','Completed','Severe headache, nausea'),
(17,17,6,'2026-09-09','09:00:00','Completed','Pain and swelling'),
(18,18,7,'2026-09-09','09:30:00','Completed','Redness, itching'),
(19,19,2,'2026-09-10','10:00:00','Completed','Dizziness, headache'),
(20,20,10,'2026-09-10','10:30:00','Completed','Abdominal pain'),
(21,21,5,'2026-09-11','11:00:00','Completed','Joint pain and stiffness'),
(22,22,8,'2026-09-11','11:30:00','Completed','Cough, sore throat'),
(23,23,9,'2026-09-12','12:00:00','Completed','Fatigue, excessive thirst'),
(24,24,4,'2026-09-12','12:30:00','Completed','Numbness, tingling'),
(25,25,1,'2026-09-13','14:00:00','Completed','Cough, chest pain'),
(26,26,10,'2026-09-13','14:30:00','Completed','High temperature'),
(27,27,6,'2026-09-14','15:00:00','Completed','Swelling, pain in limb'),
(28,28,7,'2026-09-14','15:30:00','Completed','Skin rash, itching'),
(29,29,2,'2026-09-15','16:00:00','Completed','Headache, dizziness'),
(30,30,3,'2026-09-15','16:30:00','Completed','Severe headache'),
(31,31,5,'2026-09-16','17:00:00','Completed','Joint stiffness'),
(32,32,9,'2026-09-16','17:30:00','Completed','Stomach pain'),
(33,33,10,'2026-09-17','09:00:00','Completed','Wheezing, breathlessness'),
(34,34,8,'2026-09-17','09:30:00','Completed','Fever, body ache'),
(35,35,6,'2026-09-18','10:00:00','Completed','Lower back pain'),
(36,36,9,'2026-09-18','10:30:00','Completed','Fatigue, thirst'),
(37,37,4,'2026-09-19','11:00:00','Completed','Persistent headache'),
(38,38,1,'2026-09-19','11:30:00','Completed','Chest pain'),
(39,39,5,'2026-09-20','12:00:00','Completed','Swelling, pain'),
(40,40,7,'2026-09-20','12:30:00','Completed','Itching, rash'),
(41,41,10,'2026-09-21','14:00:00','Completed','High temperature'),
(42,42,2,'2026-09-21','14:30:00','Completed','Dizziness'),
```

```
(43,43,3,'2026-09-22','15:00:00','Completed','Headache, nausea'),
(44,44,6,'2026-09-22','15:30:00','Completed','Joint pain'),
(45,45,8,'2026-09-23','16:00:00','Completed','Sneezing, cough'),
(46,46,9,'2026-09-23','16:30:00','Completed','Abdominal cramps'),
(47,47,1,'2026-09-24','17:00:00','Completed','Chest discomfort'),
(48,48,5,'2026-09-24','17:30:00','Completed','Lower back pain'),
(49,49,10,'2026-09-25','09:00:00','Completed','Fatigue, thirst'),
(50,50,7,'2026-09-25','09:30:00','Completed','Fever, weakness');

INSERT INTO Patient_Contact
(id, patient_id, contact)
VALUES
(1,1,'9800017919'),
(2,2,'9800035838'),
(3,3,'9800053757'),
(4,4,'9800071676'),
(5,5,'9800089595'),
(6,6,'9800107514'),
(7,7,'9800125433'),
(8,8,'9800143352'),
(9,9,'9800161271'),
(10,10,'9800179190'),
(11,11,'9800197109'),
(12,12,'9800215028'),
(13,13,'9800232947'),
(14,14,'9800250866'),
(15,15,'9800268785'),
(16,16,'9800286704'),
(17,17,'9800304623'),
(18,18,'9800322542'),
(19,19,'9800340461'),
(20,20,'9800358380'),
(21,21,'9800376299'),
(22,22,'9800394218'),
(23,23,'9800412137'),
(24,24,'9800430056'),
(25,25,'9800447975'),
(26,26,'9800465894'),
(27,27,'9800483813'),
(28,28,'9800501732'),
(29,29,'9800519651'),
(30,30,'9800537570'),
(31,31,'9800555489'),
(32,32,'9800573408'),
(33,33,'9800591327'),
(34,34,'9800609246'),
(35,35,'9800627165'),
(36,36,'9800645084'),
(37,37,'9800663003'),
(38,38,'9800680922'),
(39,39,'9800698841'),
(40,40,'9800716760'),
(41,41,'9800734679'),
(42,42,'9800752598'),
(43,43,'9800770517'),
(44,44,'9800788436'),
(45,45,'9800806355'),
(46,46,'9800824274'),
(47,47,'9800842193'),
(48,48,'9800860112'),
(49,49,'9800878031'),
(50,50,'9800895950');
```

```
INSERT INTO Admission
(admission_id, patient_id, room_id, admission_date, discharge_date)
VALUES
(1,1,1,'2026-09-01','2026-09-03'),
(2,2,41,'2026-09-01','2026-09-03'),
(3,3,81,'2026-09-02',NULL),
(4,4,121,'2026-09-02','2026-09-04'),
(5,5,161,'2026-09-03',NULL),
(6,6,2,'2026-09-03','2026-09-05'),
(7,7,82,'2026-09-04','2026-09-06'),
(8,8,162,'2026-09-04',NULL),
(9,9,163,'2026-09-05','2026-09-07'),
(10,10,42,'2026-09-05','2026-09-07'),
(11,11,83,'2026-09-06',NULL),
(12,12,164,'2026-09-06','2026-09-08'),
(13,13,122,'2026-09-07','2026-09-09'),
(14,14,165,'2026-09-07',NULL),
(15,15,3,'2026-09-08','2026-09-10'),
(16,16,43,'2026-09-08','2026-09-10'),
(17,17,84,'2026-09-09',NULL),
(18,18,123,'2026-09-09','2026-09-11'),
(19,19,4,'2026-09-10','2026-09-12'),
(20,20,166,'2026-09-10',NULL),
(21,21,85,'2026-09-11','2026-09-13'),
(22,22,124,'2026-09-11','2026-09-13'),
(23,23,167,'2026-09-12',NULL),
(24,24,44,'2026-09-12','2026-09-14'),
(25,25,5,'2026-09-13','2026-09-15'),
(26,26,168,'2026-09-13',NULL),
(27,27,86,'2026-09-14','2026-09-16'),
(28,28,125,'2026-09-14','2026-09-16'),
(29,29,6,'2026-09-15',NULL),
(30,30,45,'2026-09-15','2026-09-17'),
(31,31,87,'2026-09-16','2026-09-18'),
(32,32,169,'2026-09-16',NULL),
(33,33,170,'2026-09-17','2026-09-19'),
(34,34,126,'2026-09-17','2026-09-19'),
(35,35,88,'2026-09-18',NULL),
(36,36,171,'2026-09-18','2026-09-20'),
(37,37,46,'2026-09-19','2026-09-21'),
(38,38,7,'2026-09-19',NULL),
(39,39,89,'2026-09-20','2026-09-22'),
(40,40,127,'2026-09-20','2026-09-22'),
(41,41,172,'2026-09-21',NULL),
(42,42,8,'2026-09-21','2026-09-23'),
(43,43,47,'2026-09-22','2026-09-24'),
(44,44,90,'2026-09-22',NULL),
(45,45,128,'2026-09-23','2026-09-25'),
(46,46,173,'2026-09-23','2026-09-25'),
(47,47,9,'2026-09-24',NULL),
(48,48,91,'2026-09-24','2026-09-26'),
(49,49,174,'2026-09-25','2026-09-27'),
(50,50,129,'2026-09-25',NULL);

INSERT INTO Payment
(payment_id, bill_id, payment_date, amount, payment_method)
VALUES
(1,1,'2026-09-01',1750.00,'Cash'),
(2,2,'2026-09-01',1800.00,'Card'),
(3,4,'2026-09-02',1350.00,'UPI'),
(4,6,'2026-09-03',2150.00,'Insurance'),
(5,7,'2026-09-04',1750.00,'Cash'),
(6,9,'2026-09-05',2200.00,'Card'),
```

```
(7,10,'2026-09-05',1400.00,'UPI'),
(8,12,'2026-09-06',1450.00,'Insurance'),
(9,13,'2026-09-07',1350.00,'Cash'),
(10,15,'2026-09-08',2700.00,'Card'),
(11,16,'2026-09-08',1800.00,'UPI'),
(12,18,'2026-09-09',1450.00,'Insurance'),
(13,19,'2026-09-10',1700.00,'Cash'),
(14,21,'2026-09-11',1800.00,'Card'),
(15,22,'2026-09-11',1350.00,'UPI'),
(16,24,'2026-09-12',2150.00,'Insurance'),
(17,25,'2026-09-13',2200.00,'Cash'),
(18,27,'2026-09-14',2700.00,'Card'),
(19,28,'2026-09-14',1350.00,'UPI'),
(20,30,'2026-09-15',1800.00,'Insurance'),
(21,31,'2026-09-16',1750.00,'Cash'),
(22,33,'2026-09-17',1800.00,'Card'),
(23,34,'2026-09-17',1400.00,'UPI'),
(24,36,'2026-09-18',2200.00,'Insurance'),
(25,37,'2026-09-19',1400.00,'Cash'),
(26,39,'2026-09-20',2650.00,'Card'),
(27,40,'2026-09-20',1450.00,'UPI'),
(28,42,'2026-09-21',1750.00,'Insurance'),
(29,43,'2026-09-22',1800.00,'Cash'),
(30,45,'2026-09-23',1350.00,'Card'),
(31,46,'2026-09-23',1500.00,'UPI'),
(32,48,'2026-09-24',1750.00,'Insurance'),
(33,49,'2026-09-25',2200.00,'Cash');

SET FOREIGN_KEY_CHECKS = 1;

SHOW VARIABLES LIKE 'datadir';
```